Fall 2025

Software Requirements Specification

TripMate: Interactive Travel Planner

Janice Kang

Uny Li

Seliya Marahatta

Diliara Khairullina

Team A+ | Group 01

Executive Summary

Background

Travel planning has become increasingly fragmented and stressful for modern travelers. People often rely on multiple disconnected tools such as apps, spreadsheets, and group chats to organize flights, accommodations, budgets, and daily activities. This scattered approach leads to inefficiencies, inconsistent information, and unnecessary stress. Research confirms this pain point: most travelers visit numerous websites for a single trip and remain dissatisfied with the available tools. TripMate was conceived to address these challenges by creating a unified platform that simplifies planning and enhances collaboration for individuals and groups.

Description

TripMate is an AI-assisted, web-based travel planning platform designed to consolidate itinerary management, budgeting, and group collaboration into one seamless experience. The platform supports both individual and group trips, enabling members to contribute activities, edit shared plans, split expenses, and receive real-time updates. An AI recommendation module supplements the planning process by suggesting places and activities filtered by tags. The application is delivered as a responsive web app accessible across major browsers and devices.

Company Value Add

TripMate demonstrates a technically comprehensive student-driven solution that blends modern software engineering practices with realistic industry needs. The project showcases cloud deployment, secure authentication, real-time synchronization, and AI integration. For stakeholders such as educators, development teams, and potential employers, TripMate serves as evidence of strong technical capability, collaboration, and full-stack problem solving.

End-User Value Add

For travelers, TripMate simplifies planning by consolidating trip details into one organized workspace. Users benefit from clear itineraries, budget transparency, intuitive map visualization, and streamlined coordination with friends or group members. Real-time collaboration eliminates confusion caused by versioning issues and AI recommendations help reduce time spent searching for activities. The platform ultimately supports more informed decisions, better coordination, and a smoother trip planning experience.

Scope

What is Included

- User registration, login, logout, and account recovery
- Profile management and personal travel statistics
- Trip creation, editing, deletion, and daily itinerary planning
- Budget tracking with shared expense logic and per-person calculations
- Interactive map with place pinning (up to 50 pins per trip)

- Real-time collaboration (trip updates sync within 1 second)
- Group roles and permissions
- AI-based destination/activity recommendations with feedback options
- Cross-platform responsive design
- Cloud deployment, security measures, and DevOps setup

What is Not Included

- Offline mode
- Direct booking or payment processing with airlines/hotels
- Full multilingual support (initial release limited to English)
- Predictive cost forecasting or dynamic price integrations
- Automatic route optimization or navigation features
- Native mobile apps (web application only)

Justification

TripMate focuses on essential travel planning feature that aligns with the project's timeframe, resource limits, and educational goals. The included features represent the minimum viable product required to demonstrate full stack development., collaboration tools, and AI integration. Excluded items require additional APIs, complex optimization algorithms, or extended development time that exceeds the course constraints. By prioritizing a clear, functional, and cohesive set of capabilities, the project ensures feasibility while delivering meaningful value to users.

BACKGROUND **1**

DESCRIPTION **1**

COMPANY VALUE ADD 1
END-USER VALUE ADD 1

SCOPE **1**

WHAT IS NOT INCLUDED 2

JUSTIFICATION **2**

1.1 DOCUMENT AUTHORS **7**

1.2 DOCUMENT REVISION HISTORY **7**

1.3 DOCUMENT PURPOSE **7**

1.4 AUDIENCE **7**

1.5 GROUP AGREEMENT **8**

Team #01 ..... *8*
TripMate: Interactive Travel Planner *8*
Project Time Frame *8*
Team Members *8*
Team Leadership *8*
Team Functions/Roles *8*
Team Meetings *8*
Team Problems *8*
Team Commitment *9*

2.1 PROJECT PROPOSAL **10**

2.1.1 PROJECT BACKGROUND 10
2.1.2 PROBLEM STATEMENT 10
2.1.3 PRODUCT VISION 10

2.2 STAKEHOLDERS AND USERS **10**

Internal Stakeholders *10*
External Stakeholders *11*

2.3 PROJECT SCOPE **11**

2.3.1 PROJECT OBJECTIVES 11
2.3.2 DELIVERABLES 11
2.3.3 PROJECT BOUNDARIES 12
In Scope: *12*
Out of Scope: *12*
2.3.4 PROJECT CONSTRAINTS 12
2.3.5 PROJECT ASSUMPTIONS 12
2.3.6 RESOURCE REQUIREMENTS 12
2.3.7 QUALITY STANDARDS 12
2.3.8 SUCCESS CRITERIA 13
Approval Criteria *13*
Exit Criteria *13*

2.3.9 Change Control Process 13

2.4 SYSTEM RISKS 13

1. Technical Risks 13
2. Resource Risks 14
3. Scope Risks 14
4. Adoption Risks 14

2.5 OPERATING ENVIRONMENT 14

2.5.1 User Environment 14
2.5.2 Development Environment 15

2.6 FUNCTIONAL REQUIREMENTS 15

2.7 NONFUNCTIONAL REQUIREMENTS 17

2.7.1 Operational 17
2.7.2 Performance 17
2.7.3 Security 17
2.7.4 Cultural and political 18
2.7.4.1 Language and Localization 18
2.7.4.2 Cultural Sensitivity 18
2.7.4.3 Legal and Political Compliance 18
2.7.4.4 Accessibility and Inclusivity 18

2.8 UI/UX INTERFACE MOCK-UPS 19

Landing page 19
Sign up page 20
Login page 21
Account Recovery Page 22
Home page 23
Destination Management page 24
Itinerary Management page 25
Itinerary Management page / AI Assist pop-up 26
Budget Management page 27
Budget Management page / Shared Expense Details view 28
Group Management page 29
Group Management Page / Invite Traveler Form 30
Profile Page 31

3.1 DATA FLOW DIAGRAMS 32

3.2 USER STORIES AND RELATED USE CASE SCENARIOS 32

User Story 1 – Account & Access Management 32
User Story 2 – Trip Creation & Personal Planning 33
User Story 3 – Group & Collaboration Management 35
User Story 4 – Smart Recommendations (AI Module) 37

3.3 ACTIVITY DIAGRAMS 39

AD1 Account & Access Management 39
AD2 Trip Creation & Individual Traveler Trip Planning 40

AD3 Group & Collaboration Management 41
AD3.1 Trip Manager Perspective *41*
AD3.2 General Traveler Perspective *42*
AD4 Smart Recommendations (AI Module) 43
AD5 Editing Existing Trip 44
3.4 BUSINESS RULES 45
5.1 ENTITY RELATIONSHIP DIAGRAM 48
5.2 DATA DICTIONARY **48**
5.3 SQL TABLES & ATTRIBUTES 53
5.3.1 Users 53
5.3.2 Profiles 53
5.3.3 Trips 54
5.3.4 TripMembers 55
5.3.5 Invitations 55
5.3.6 Destinations 56
5.3.7 PlacePins 56
5.3.8 ItineraryDays 56
5.3.9 ItineraryItems 57
5.3.10 Expenses 57
5.3.11 ExpenseShares 58
5.3.12 RecommendationRequests 58
5.3.13 Recommendations 58
5.3.14 Ratings 59
5.3.15 Notifications 59
5.3.16 AuditLogs 60
5.4 RELATIONSHIPS SUMMARY 60
6.1 WORK BREAKDOWN STRUCTURE 62
Level 1: Product Goal 62
Level 2: Major Deliverables (High-Level Features/Epics) 62
Level 3: Iteration/Sprint-Level Breakdown (User Stories/Features) 62
Level 4: Sprint Tasks (Granular WBS) 64
How This Works in Agile 68
6.2 MILESTONES & ACCEPTANCE CRITERIA 69
7.1. USER AUTHENTICATION & PROFILE MANAGEMENT 71
7.2. TRIP & ITINERARY MANAGEMENT 73
7.3. BUDGET & EXPENSE TRACKING 75
7.4. INTERACTIVE TRIP MAP & PLACES PINNING 77
7.5. GROUP COLLABORATION 78
7.6. AI-POWERED RECOMMENDATIONS 80
7.7. PERSONAL PLACES TRACKING & STATISTICS 81
7.8. CROSS-PLATFORM USABILITY 82

7.9. DEPLOYMENT, SECURITY & DEVOPS 83

7.10. ADDITIONAL TECHNICAL & SUPPORTING STORIES 85

Section 1

1.1 Document Authors

Janice Kang – Backend Developer
Uny Li – Cloud & Security Engineer
Seliya Marahatta – AI & Data Analytics Engineer
Diliara Khairullina – Project Manager & Frontend Developer

1.2 Document Revision History

WEEK	DATE	Revisions
3	September 21, 2025	• Section 1 • Section 2.1
4	September 28, 2025	• Sections 1.1, 1.4, 2.1 (updated) • Sections 2.2-2.5
5	October 5, 2025	• Section 2.6-7
6	October 14, 2025	• Section 2.6 (updated) • Section 2.8
7	October 26, 2025	• Section 3
8	October 28, 2025	• Section 2.8 (updated)
9	November 5, 2025	• Section 4
10	November 16, 2025	• Section 5
11	November 23, 2025	• Section 3 (use case description added) • Section 6
12	November 30, 2025	• Sections 7, 8
13		•
14		•

1.3 Document Purpose

This document explains all the requirements for TripMate, our AI-assisted travel planner. It shows what the system should do, how it should work, and what features it should have. The purpose is to help our team, our professor, and any other stakeholders understand the project clearly. It will guide us when designing, coding, and testing TripMate. By defining requirements in detail, the document aims to reduce misunderstandings and guide the project toward successful delivery.

1.4 Audience

This document is intended for our project team (including Backend Developer, Cloud & Security Engineer, AI & Data Analytics Engineer, and Project Manager & Frontend Developer), who will use the requirements as a guide for the design, development, and testing of TripMate. It helps the team understand what features are needed, how the system should behave, and what goals must be achieved. It is also written for our professor and evaluators, who will review how well the system is defined, organized, and planned, and assess the project's alignment with course requirements. In addition, the document is relevant for

potential end users such as travelers, influencers, and tourism businesses, who may want to understand the features and goals of TripMate and how it can support their travel planning.

1.5 Group Agreement

Team #01

TripMate: Interactive Travel Planner

Project Time Frame

September 2025 – April 2026

Team Members

- Janice Kang
- Uny Li
- Seliya Marahatta
- Diliara Khairullina

Team Leadership

Diliara Khairullina

Team Functions/Roles

- Janice Kang: Backend Development (APIs, database design, authentication)
- Uny Li: Cloud & Security (deployment, cloud infrastructure, data protection)
- Seliya Marahatta: AI & Data Analytics (recommendations and data analysis)
- Diliara Khairullina: Project Management & Frontend Development (UI/UX, integration, timelines)

Team Meetings

- Weekly team meetings on Wednesdays at 1:30 PM via Microsoft Teams and/or in-person
- Additional meetings scheduled as needed for milestone reviews or urgent issues

Team Problems

As the project is still in its early stages, our team has identified potential challenges that may impact progress. Stakeholder misalignment could create uncertainty about priorities and expectations if goals are not clearly communicated. Scope creep is also a risk, as new ideas and features may be added beyond the original plan, making it harder to maintain focus. Furthermore, some requirements remain ambiguous or incomplete, which may require ongoing clarification and refinement.

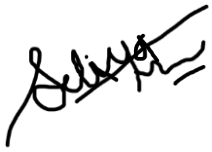
We also anticipate possible issues such as technical knowledge gaps, since not all members are fully familiar with tools like API integration, design platforms, or cloud and AI tool. In addition, conflict resolution may become necessary, as different perspectives on design and functionality will likely require open discussion and compromise.

Team Commitment

Each team member agrees to:

- Participate fully in all project activities, contributing ideas, effort, and time to achieve team goals.
- Participate professionally, communicating respectfully, supporting team members, and maintaining a positive working environment.
- Meet responsibilities, completing assigned tasks on time and to the best of their ability.
- Give appropriate credit to other team members for their contributions, ensuring fair recognition of work.
- Take shared responsibility for the quality and completeness of all deliverables, understanding that the success of TripMate depends on both individual and team efforts.

The undersigned members agree to work together on the project until the end of the PRJ666 next Semester. They recognize that as a team and individually they are equally responsible for the quality of all deliverables.

Name	**Date**	**Signature**
Janice Kang	2025-09-21	
Uny Li	2025-09-21	
Seliya Marahatta	2025-09-18	
Diliara Khairullina	2025-09-21	

Section 2

2.1 Project Proposal

2.1.1 Project Background

Travel planning has become a fragmented, time-consuming process where travelers juggle multiple apps, spreadsheets, and group chats to organize flights, accommodations, budgets, and daily activities. Industry data confirms this pain point: a 2025 YouGov survey reported that 70% of U.S. vacation bookers find the experience stressful, while an Accenture study revealed that most travelers visit up to ten different websites for a single trip and remain dissatisfied with the tools available.

The idea for TripMate grew from the team's shared frustrations in planning student trips on tight budgets. It was conceived as a unified workspace that brings all essential planning functions together in one coordinated environment.

2.1.2 Problem Statement

Travelers often struggle to organize trip details because they must rely on multiple disconnected tools for activities, expenses, and group coordination. As a result, information becomes inconsistent, updates are hard to share, and travelers lose control over budgets and schedules, leading to unnecessary stress and inefficiency.

The core problem is the absence of a cohesive system that maintains the relationship between planning decisions, financial tracking, and group input. TripMate addresses this gap by creating a synchronized environment where all trip elements are inherently connected and updated in real-time.

2.1.3 Product Vision

TripMate is envisioned as a comprehensive, AI-assisted travel planning platform that transforms the way individuals and groups organize trips. The product's vision is to eliminate the current "scavenger hunt" approach to travel planning by providing a single, interactive, and collaborative environment.

Rather than simply combining features, TripMate will create an intelligent ecosystem where users will be able to add destinations, activities, and schedules, track or split expenses, and receive suggestions that help choose places, organize daily plans, and enhance overall travel experiences. Our vision is a reliable, user-centered tool that makes trip planning simpler, so travelers spend less time coordinating and more time experiencing their journeys.

2.2 Stakeholders and Users

Internal Stakeholders

- **Project Manager**
  Oversees project coordination, manages timelines, and ensures smooth integration between frontend and backend development.
- **Backend Developer**
  Responsible for building APIs, handling authentication, and designing the database.

- **AI & Data Analytics Engineer**
  Develops AI modules for recommendation engines.
- **Cloud & Security Engineer**
  Manages cloud infrastructure, deployment, security protocols, and ensures system scalability and data privacy.
- **UI/UX Designer (Role to be assigned or covered by Frontend Developer)**
  Designs user interfaces and ensures the app is user-friendly and responsive across devices.
- **Quality Assurance/Testers (Role to be assigned)**
  Responsible for testing the system, identifying bugs, and verifying that features meet requirements.

External Stakeholders

- **Travelers / End Users**
  Individuals or groups who use TripMate to plan, budget, and organize trips collaboratively.
- **Travel Bloggers / Influencers**
  Users who contribute travel suggestions, create and share itineraries, and promote destinations.
- **Tourism Businesses**
  Hotels, restaurants, tour operators, and other service providers that may benefit from increased visibility and bookings via TripMate.
- **Third-Party API Providers**
  Providers of map services, travel data, budgeting tools, and other external APIs integrated into TripMate.
- **Regulatory Bodies / Data Privacy Authorities**
  Entities ensuring that the platform complies with data protection laws and travel-related regulations.

2.3 Project Scope

2.3.1 Project Objectives

The project will deliver a browser-based web application that consolidates itinerary planning, possibility of shared budgeting, and AI recommendations so trip can be planned in one place. We will leverage cloud hosting and modern web tooling to improve collaboration and clarity. The key deliverables will be stated in the following section.

2.3.2 Deliverables

- Itinerary builder for destinations, schedules, and activities
- Budget tracker with expense split & settlement
- Interactive trip map with place pinning and manual sequencing
- Real-time collaborative trip boards with live editing and role-based access
- Lightweight AI recommendations for places
- Secure authentication and encrypted data storage

- Responsive design for desktop and mobile browsers

2.3.3 Project Boundaries

In Scope:

- Integration of itinerary, budgeting, mapping, and collaboration features
- Equal-share expense splitting across selected users
- Tag-based place recommendations
- Cloud deployment and data security compliance

Out of Scope:

- Offline functionality (platform requires internet access)
- Direct booking or payment processing with third-party platforms (future enhancement)
- Full multi-language support (initial release in English only)
- Predictive cost forecasting and live price integrations (future enhancement)
- Automatic route optimization

2.3.4 Project Constraints

- Limited budget and timeline across two semesters
- Dependence on external APIs/tiles for maps (usage limits/costs)
- Team members' varying experience with AI, cloud, and APIs
- Performance target: support at least 50 concurrent users at launch

2.3.5 Project Assumptions

- Users have internet access and basic digital literacy
- Required third-party APIs remain available and stable
- AI features will improve with user feedback
- Collaboration features will start simple and improve incrementally

2.3.6 Resource Requirements

- **Human Resource:** Backend & Frontend Developer, AI/Data Engineer, Project Manager, Cloud/Security Engineer
- **Technical Resources:** Access to cloud infrastructure (e.g., AWS, Azure), third-party map APIs/tiles
- **Tools**: Development frameworks (React, Node.js, Python/AI libraries), collaboration tools (GitHub)
- **Budget:** Cloud hosting and API usage costs

2.3.7 Quality Standards

- Usability: Intuitive and responsive user interface across devices
- Performance: Support at least 50 concurrent users with low latency
- Security: Encrypted user data, secure authentication, compliance with privacy standards
- Accuracy: Reliable budget tracking
- Collaboration: Real-time synchronization with minimal conflicts

2.3.8 Success Criteria

Approval Criteria

- Successful completion and demonstration of all MVP features
- Positive user feedback from test groups on usability and collaboration
- AI recommendations provide at least 3 useful suggestions for places or activities per trip
- System passes security and performance tests under expected load
- Final approval from faculty or client

Exit Criteria

- All deliverables completed and tested according to approval criteria
- Final project documentation, including user manuals and technical specs, submitted
- Handover of source code repository and cloud infrastructure credentials
- Project review meeting conducted with stakeholders
- Official project closure and sign-off

2.3.9 Change Control Process

- All change requests must be submitted via the Teams channel
- All changes must be evaluated by the project lead and team for impact on scope, time, and resources
- All changes must be approved by the majority of team members before implementation
- All changes must be documented, and the project scope must be updated accordingly

2.4 System Risks

Developing TripMate involves a range of risks that may affect the project's success. These risks fall into several categories: technical, resource-related, scope-related, and adoption. Each risk must be carefully considered to reduce the chance of delays, cost overruns, or functionality issues.

1. Technical Risks

- **API Limitations**
  Risk: TripMate relies on third-party APIs (e.g., maps, data and budgeting services). These APIs may impose cost limits, change terms of service, or restrict access, which could disrupt system functionality.
  Impact: Medium to high – loss of core features (maps, cost estimates) would reduce system value.
  Mitigation: Begin with free/basic APIs, design systems with modular API integration so alternatives can be swapped in, and document dependencies early.

- **Complex AI Integration**
  Risk: AI modules for recommendations require advanced knowledge and may not perform as expected. Poor results could reduce user trust.
  Impact: Medium – delays or reduced quality of AI features.
  Mitigation: Start with simple rule-based recommendations for MVP, then iterate with more complex AI. Leverage existing AI libraries and gradually refine algorithms with user feedback.

- **Real-Time Collaboration Challenges**
  Risk: Supporting multiple users editing the same trip simultaneously can lead to synchronization conflicts, data overwrites, or performance bottlenecks.
  Impact: High – collaboration is one of TripMate's unique features.
  Mitigation: Investigate proven collaboration models (e.g., version control, operational transforms), prototype early, and stress-test with simulated concurrent users.

2. Resource Risks

- **Skill Gaps in Team**
  Risk: Team members may lack prior experience with AI, cloud deployment, or API systems.
  Impact: Medium – slower progress, potential errors.
  Mitigation: Assign research and training tasks early, share knowledge within the team, and leverage tutorials and open-source communities.

- **Time Constraints**
  Risk: Limited semester-based schedule may not allow full implementation of all planned features.
  Impact: High – risk of incomplete product at demo stage.
  Mitigation: Prioritize minimum viable product (MVP) first (itinerary + budget tracker), with advanced features (AI, group sync) added in later sprints.

3. Scope Risks

- **Scope Creep**
  Risk: New ideas and an excess of features could expand project scope beyond what is feasible within time and resource limits.
  Impact: Medium to high – delays, incomplete or buggy system.
  Mitigation: Use a change control process, maintain a clear roadmap, and classify advanced features as "future enhancements" rather than MVP requirements.

4. Adoption Risks

- **Competition from Established Apps**
  Risk: Existing solutions (e.g., TripIt, Wanderlog) already cover parts of the travel-planning process. If TripMate fails to differentiate, users may prefer existing tools.
  Impact: Medium – could affect project's perceived value and user engagement.
  Mitigation: Focus on unique strengths such as integrated budgeting + AI recommendations and real-time group collaboration, which competitors handle poorly. Highlight these features in testing and presentations.

2.5 Operating Environment

2.5.1 User Environment

- Platforms: Web application
- Hardware: Smartphones and personal computers
- Operating Systems: Windows, macOS
- Browsers: Chrome, Firefox, Safari

2.5.2 Development Environment

- IDE/Code Editors: Visual Studio Code
- Version Control: Git with GitHub for collaboration and code management
- Languages & Frameworks:
    - Frontend: HTML5, JavaScript, React
    - Backend: Node.js
- Database: MySQL for structured storage
- APIs/Services: Integration with third-party APIs (Google Maps API)

2.6 Functional Requirements

TripMate's features are designed around four key personas: **Single Traveler**, **Trip Managers, Travel Group, and General Travelers**. Each persona interacts with the system according to their goals and responsibilities. Single Travelers plan and manage their own trips independently, Trip Managers coordinate group trips and oversee budgets, Travel Group Members contribute activities and track personal expenses, and General Travelers collaborate without a designated manager, with every participant able to create, edit, and manage trip details. These personas guide how core features – such as itinerary management, budgeting, collaborative boards, interactive maps, and AI recommendations – are designed and used.

Each requirement below includes a **MoSCoW priority** (Must, Should, Could, Won't) and, where possible, measurable or testable **acceptance criteria** for validation.

- **Itinerary Management**
  Priority: Must Have
    - Single Traveler/Trip Manager/General Travelers: Create, edit, and delete itineraries
      Acceptance: System confirms creation, edit, or deletion within 2 seconds, displaying the updated itinerary list
    - All Users: Add destinations, activities, and schedules
      Acceptance: System saves new activities instantly; itinerary reloads with changes visible to all collaborators within 1 second
    - Single Traveler/Trip Manager/General Travelers: Reorder or sequence trip events manually
      Acceptance: Users can drag and drop items; the system maintains order on reload and in shared views

  Limit: Each itinerary supports up to 100 scheduled activities to ensure responsiveness
- **Budget & Expense Tracking**
  Priority: Must Have
    - Single Traveler/Trip Manager/General Travelers: Add expenses (flights, hotels, food, etc.)
      Acceptance: Expense entries reflect on-screen within 2 seconds
    - Trip Manager/General Travelers: Split expenses among group members
      Acceptance: Balances automatically calculated to a precision of $0.01
    - All Users: Track balances
      Acceptance: Changes appear in the total budget view immediately for all collaborators

  Limit: Each trip can include up to 100 expense items to maintain system responsiveness

- **Interactive Trip Map**
  Priority: Must Have
  - All Users: Pin locations (restaurants, attractions, hotels) on trip map
    Acceptance: System confirms pin placement instantly and displays the new marker within 2 seconds of user action
  - All Users: View trip stops on map
    Acceptance: All pinned locations are visible within the current itinerary map view; loading time does not exceed 3 seconds for up to 50 pins

  Limit: Each trip may include up to 50 pinned locations to preserve load performance

- **Collaboration Features**
  Priority: Should Have
  - Trip Manager/General Travelers: Invite users & assign them role (e.g., viewer/editor)
    Acceptance: The system successfully sends an invitation email or link within 10 seconds of the request. The invited user can join the trip and access only the functions associated with their assigned role
  - Travel Group/General Travelers: Get real-time trip board updates
    Acceptance: Edits by one user appear on other users' screens within 1 second
  - Trip Manager/Travel Group/General Travelers: Edit trip boards collaboratively with role-based access
    Acceptance: When two or more users edit the same section, the latest saved version is displayed
- **AI Recommendations**
  Priority: Should Have
  - All Users: Get recommendations of destinations or activities based on tags (e.g., "food," "attractions")
    Acceptance: System generates at least three relevant suggestions within 3 seconds of the request
    Data Source: AI model uses integrated, publicly available travel data (e.g., Google Maps API, popular travel sources) to generate recommendations
    Feedback: Users can rate or mark suggestions as "useful" or "not relevant," enabling future refinement
    Reliability: Recommended items must match tag categories with at least 80% relevance accuracy during testing
- **User Accounts & Authentication**
  Priority: Must Have
  - All Users: Register/login/logout
    Acceptance: Registration completes successfully when all required fields are filled, login grants access to the dashboard within 3 seconds after valid credentials are entered, logout immediately ends the active session and redirects to the login page within 2 seconds
  - All Users: Secure account management
    Acceptance: Passwords are stored using secure hashing; users are logged out after 30 minutes of inactivity
  - All Users: Account recovery via secure email verification

Acceptance: Reset link delivered within 60 seconds of request

- **Cross-Platform Usability**
  Priority: Must Have
  - All Users: Responsive design (desktop & mobile browser access)
    Acceptance: All primary functions (login, itinerary view, budgeting) remain usable on mobile at screen widths ≥ 360px
    Supported Browsers: Chrome, Edge, Safari, Firefox
    Consistency: If multiple browser tabs are open for the same user, updates must remain synchronized within 1 second

2.7 Nonfunctional Requirements

2.7.1 Operational

- The system must be available 24/7, excluding scheduled maintenance.
- The system must operate on modern web browsers (Chrome, Edge, Safari, Firefox) without requiring additional plugins.
- The system must support responsive design, ensuring usability across desktop and mobile devices.
- The system must provide automatic backups of user data daily.

2.7.2 Performance

- The system must support at least 50 concurrent users with no noticeable latency during editing or collaboration.
- The system must return responses to user actions (e.g., adding expenses, editing itineraries) within 2 seconds under normal load.
- The system must synchronize real-time collaboration updates within 1 second of a change being made.
- The system must scale to support up to 500 users during peak travel seasons with minimal degradation in performance.

2.7.3 Security

- The system must require users to authenticate with a unique email and password before accessing any features.
- The system must enforce secure password storage using hashing and salting.
- The system must implement role-based access control to restrict access to sensitive actions.
- The system must encrypt all sensitive data (e.g., user credentials, expenses) at rest and in transit using industry-standard protocols.
- The system must ensure that financial and personal information is accessible only to authorized users within a trip group.
- The system must protect against common web security threats.
- The system must restrict access to third-party API credentials, so they are never exposed to client-side users.

- The system must log authentication attempts and unusual activities for auditing and monitoring.
- The system must provide account recovery mechanisms through secure email verification.

2.7.4 Cultural and political

2.7.4.1 Language and Localization

- The system must support English language.
- Date, time, and currency formats must adapt to the user's region.

2.7.4.2 Cultural Sensitivity

- Content, images, and recommendations must be culturally appropriate for each region.
- No content should promote activities illegal or culturally offensive in the user's country.

2.7.4.3 Legal and Political Compliance

- The system must comply with local data privacy laws, including GDPR in the EU and PIPEDA in Canada.
- Certain travel recommendations must adhere to political restrictions.
- Payment processing must comply with local financial regulations.

2.7.4.4 Accessibility and Inclusivity

- The system must provide accessibility features for users with disabilities in compliance with local laws.
- Holiday and festival notifications should respect local cultural holidays.

2.8 UI/UX Interface Mock-ups

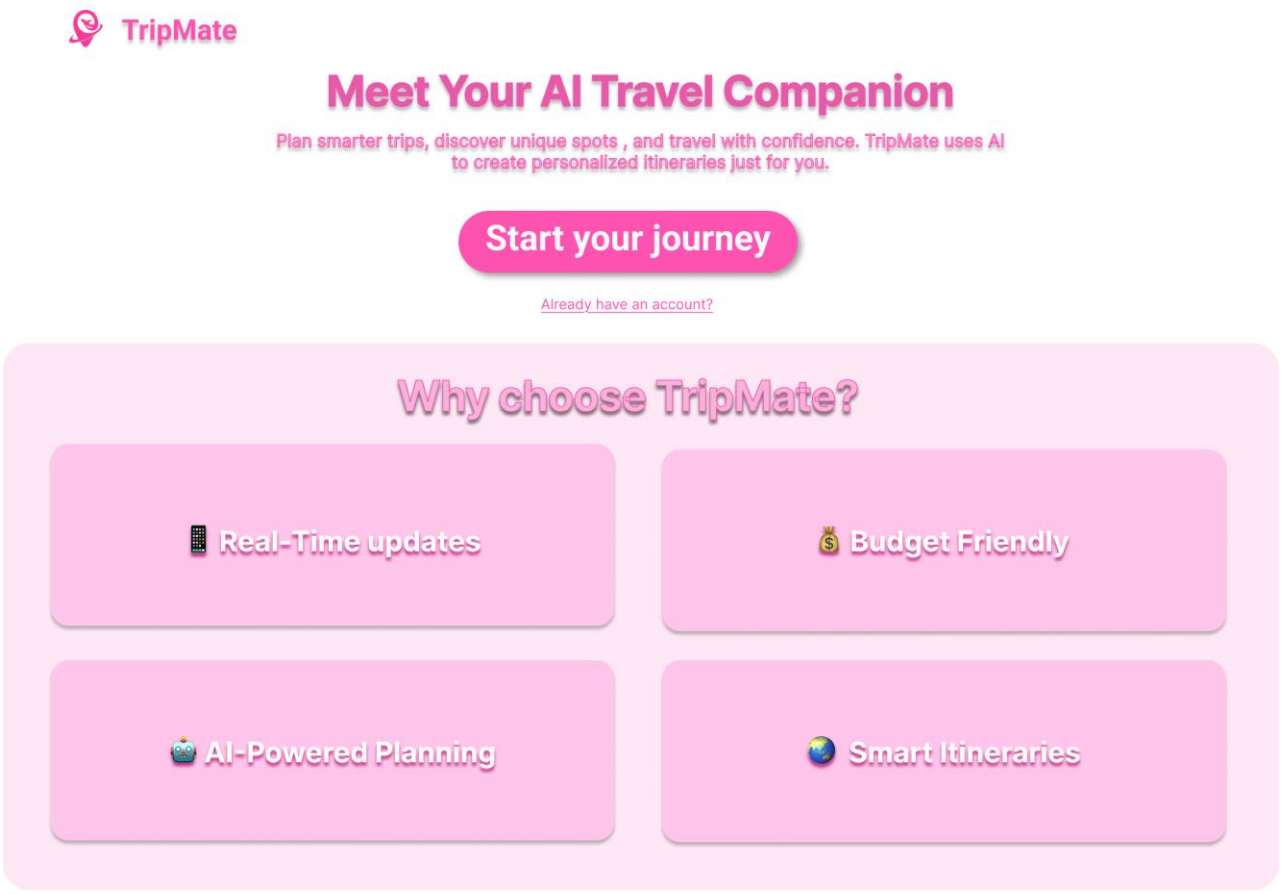
Link to Figma design page: [Figma Design - Team 1]

Landing page

Purpose: Welcomes users to TripMate and introduces the app's key features, encouraging them to explore or sign up.

Key Functions:

- Log In links for easy navigation
- Eye-catching call-to-action button "Start your journey"
- Highlighted features and benefits of TripMate

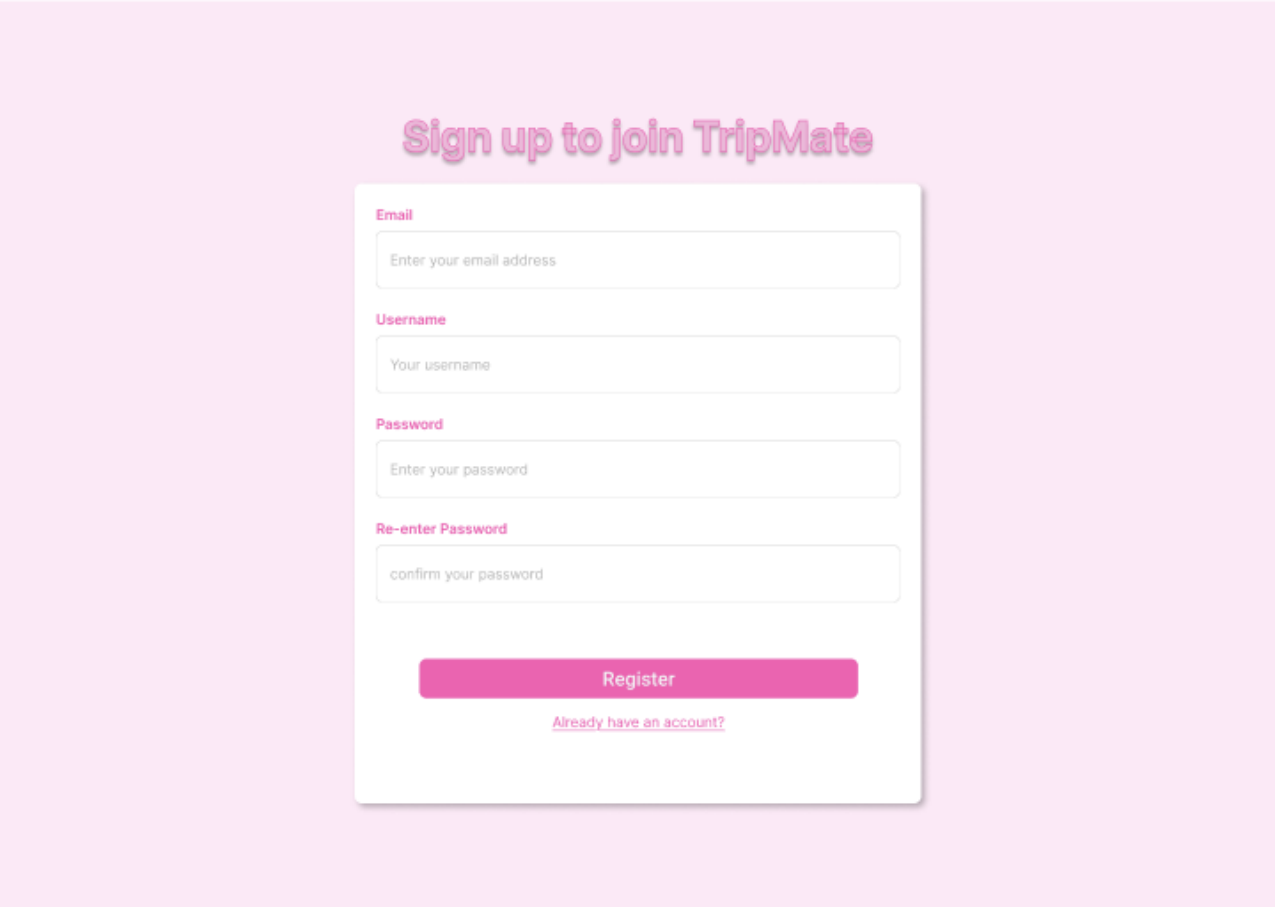


Sign up page

Purpose: Allows a new user to register for a TripMate account by entering name, email, and password.

Key Functions:

- Input fields for user information
- Password confirmation and validation
- “Register” button to submit form
- “Log In” link for existing users

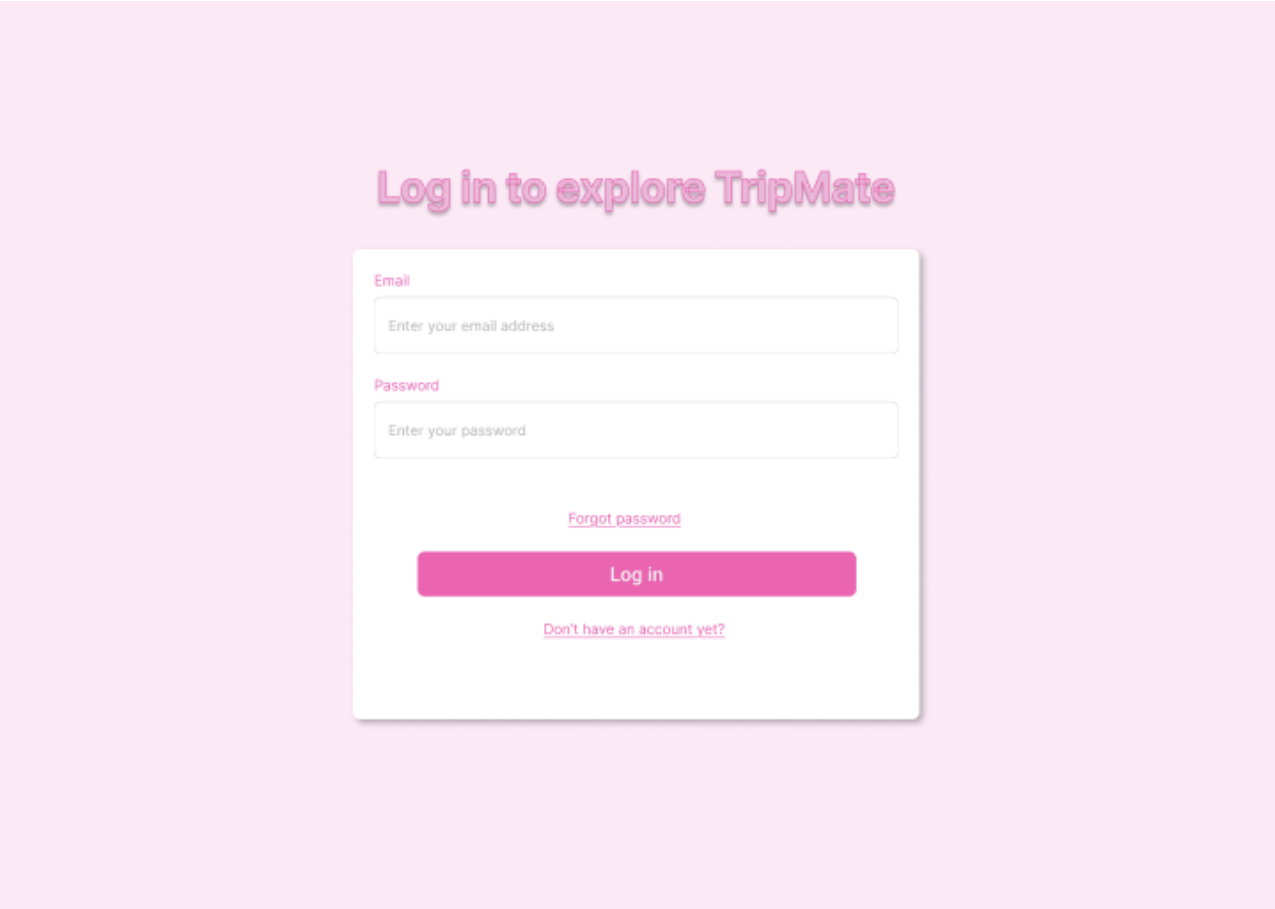


Login page

Purpose: Authenticates existing users to access the main TripMate dashboard.

Key Functions:

- Email and password input
- “Forgot Password” link
- “Sign Up” button for new users

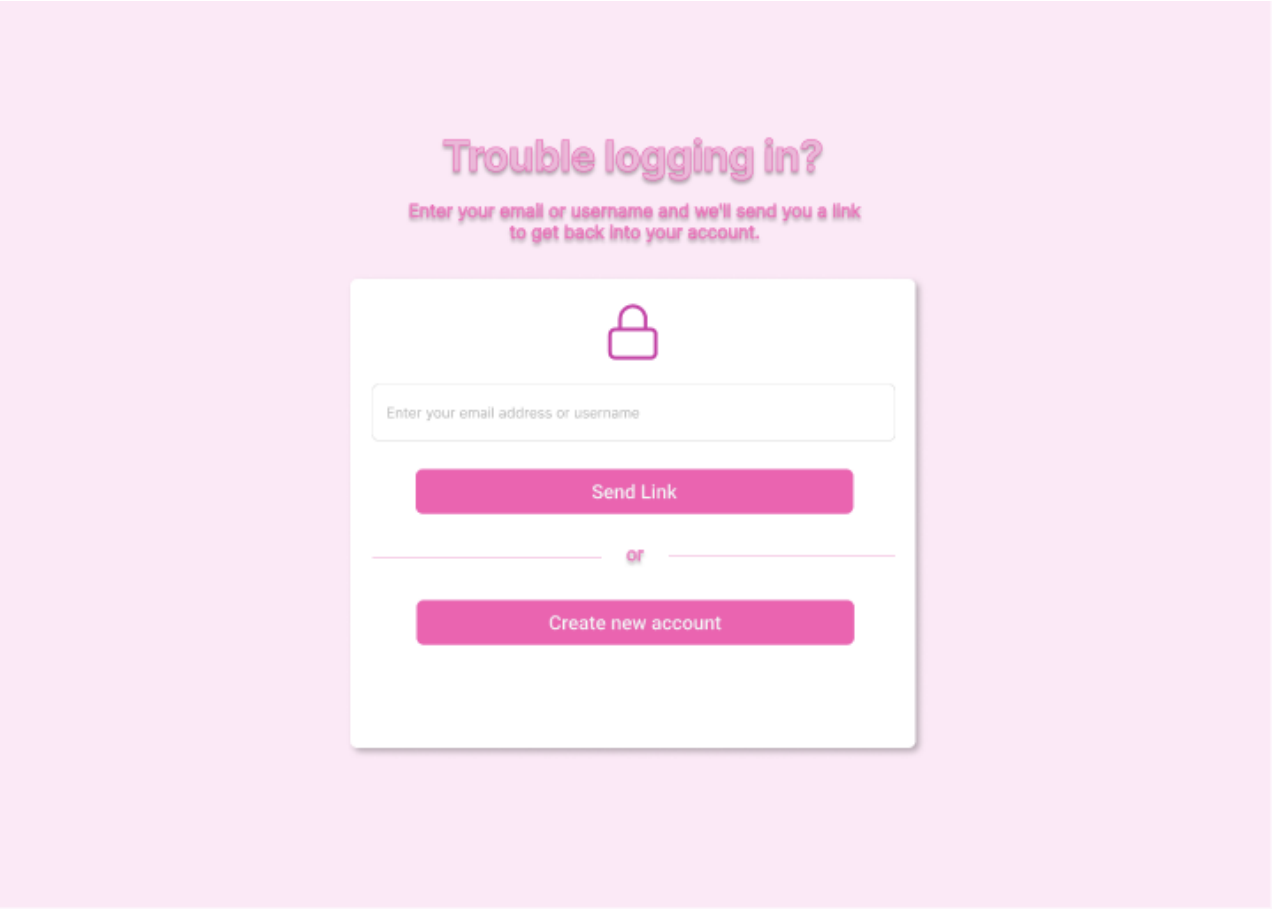


Account Recovery Page

Purpose: Allows users who have forgotten their password to regain access to their TripMate account.

Key Functions:

- Input field for username or email
- “Send Recovery Link” button to email the password reset link
- “Create Account” link for users who don’t have an account yet

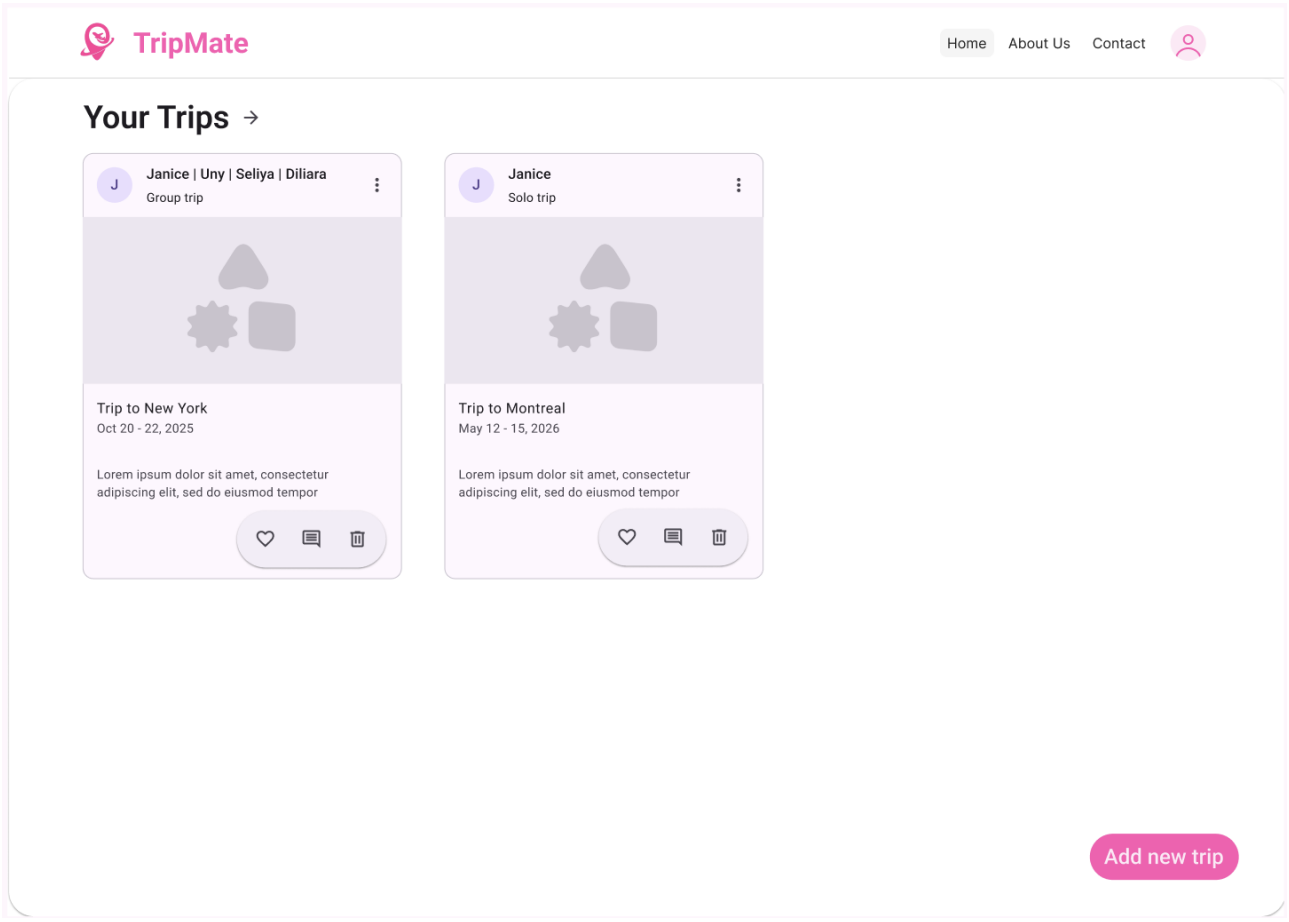


Home page

Purpose: Displays all trips created by the user and provides quick access to trip management tools.

Key Functions:

- Trip cards showing name, date, and settings
- “Add New Trip” button to create a new itinerary
- Navigation bar for accessing different modules

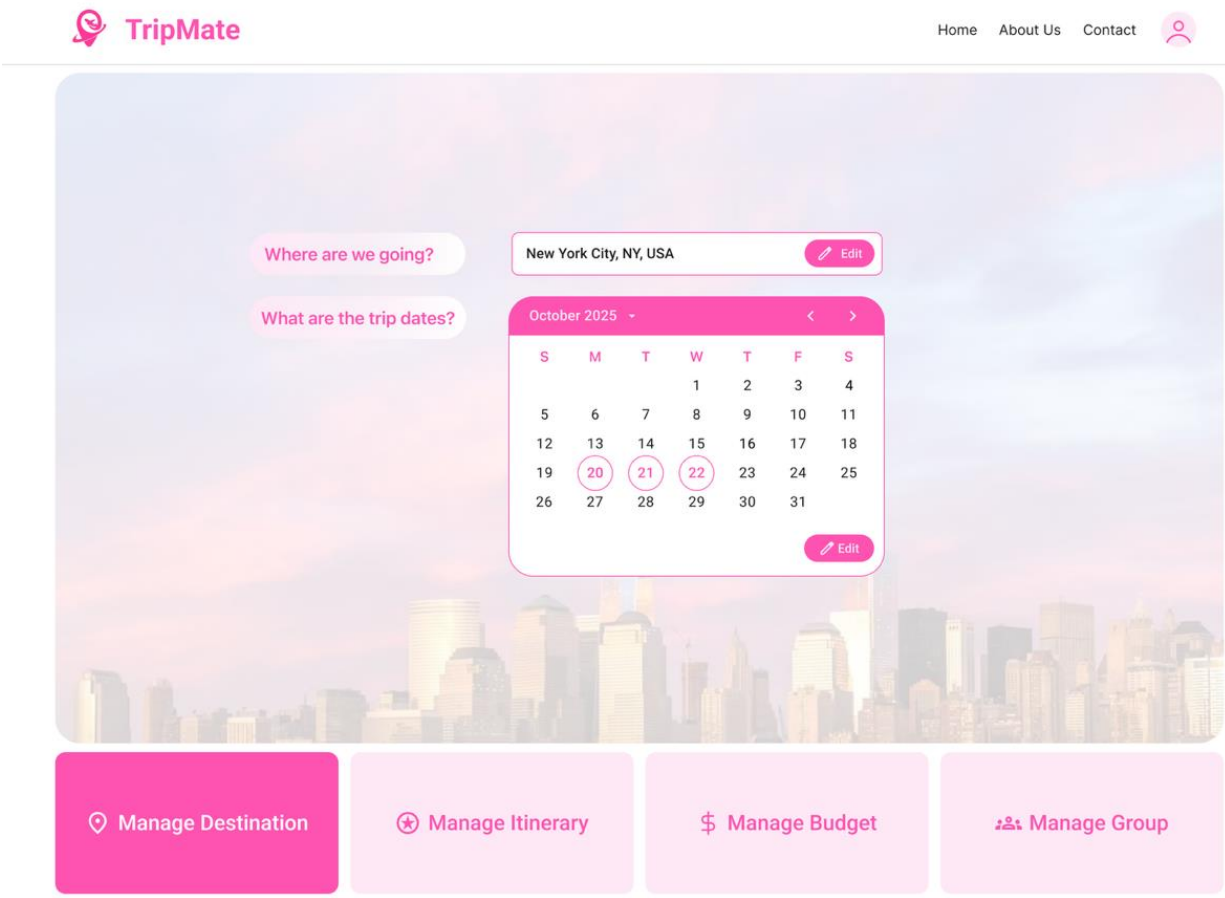


Destination Management page

Purpose: Lets users select and update destinations and trip dates.

Key Functions:

- Calendar for choosing travel dates
- Input for city or country
- “Manage Itinerary,” “Manage Budget,” and “Manage Group” navigation buttons

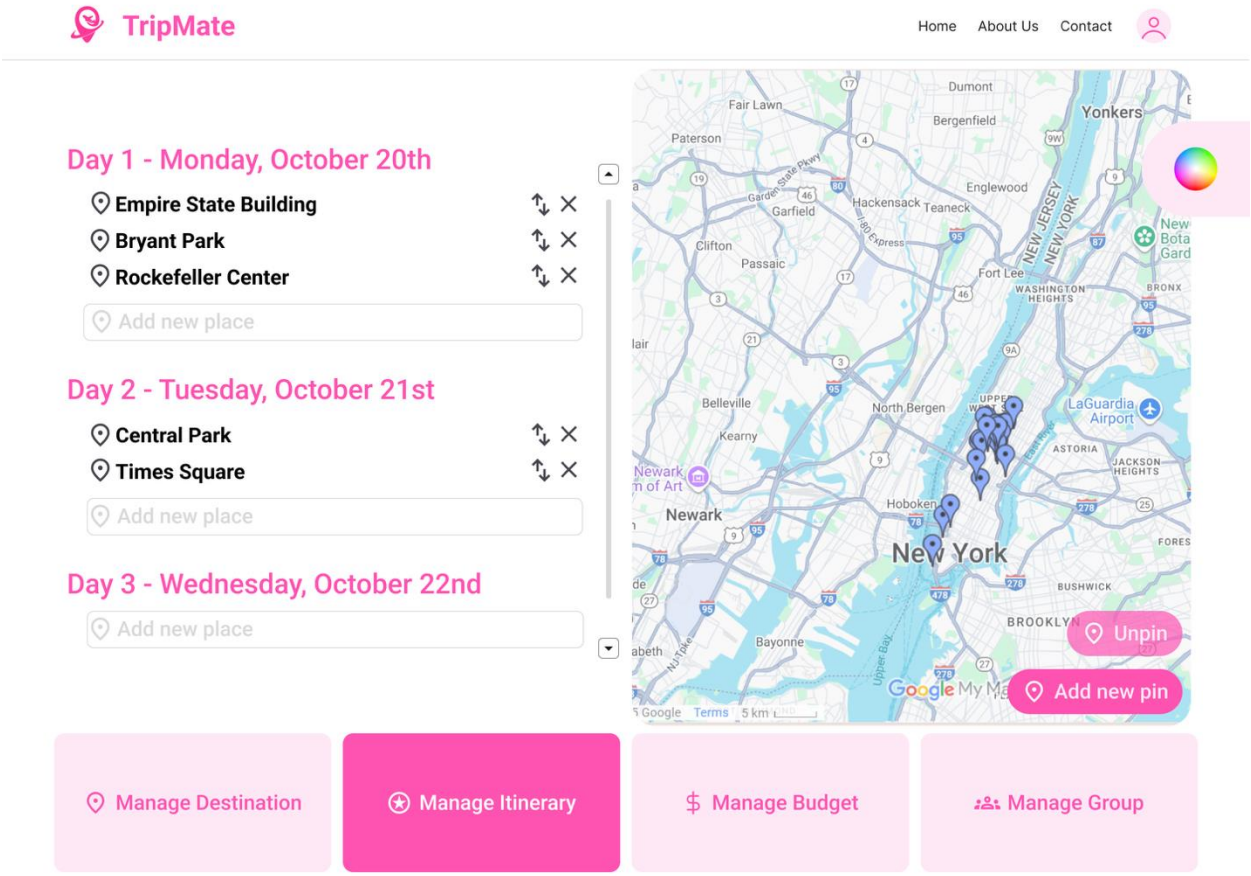


Itinerary Management page

Purpose: Helps users plan daily schedules with location and activity details.

Key Functions:

- Editable day-by-day itinerary
- Integrated map view
- Buttons to add, remove, or reorder destinations
- Button to access AI Assist for places recommendations

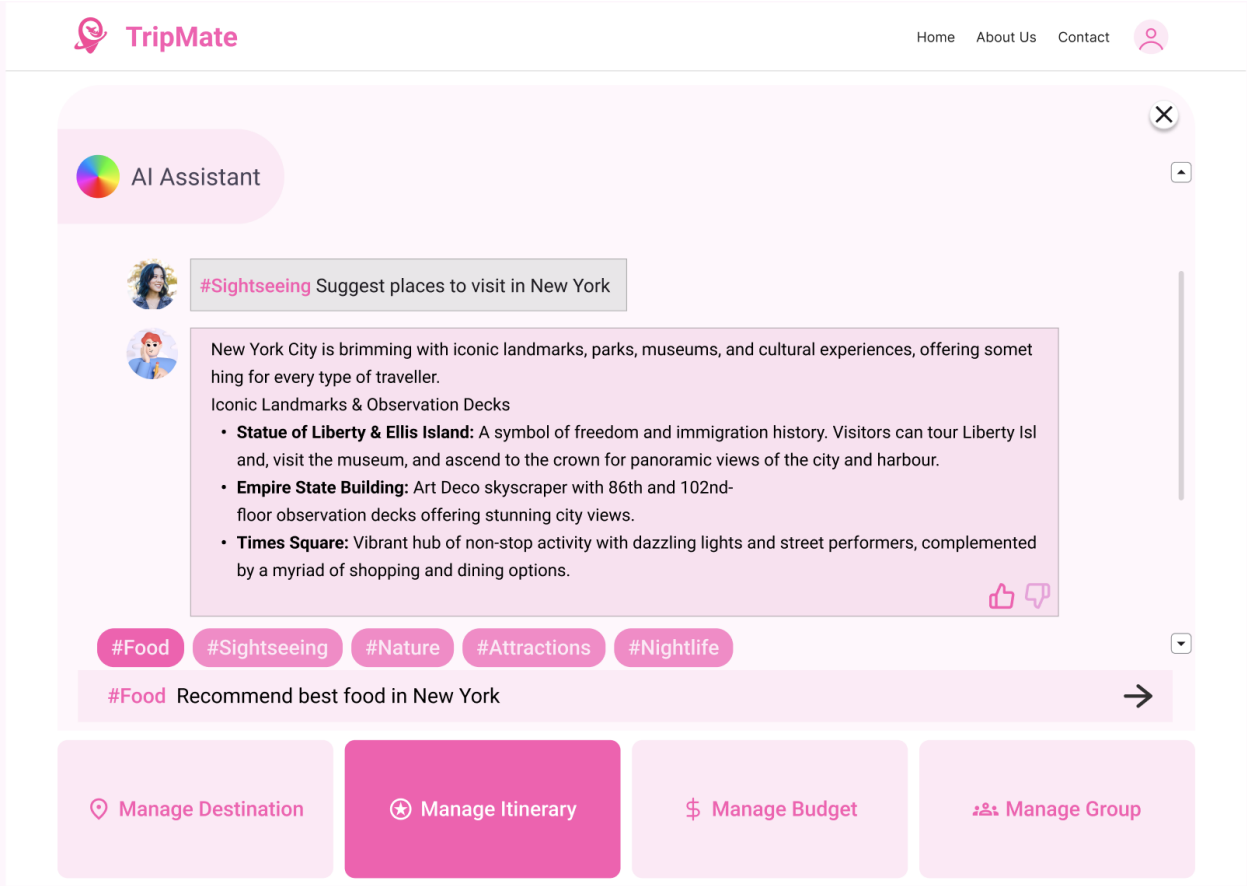


Itinerary Management page / AI Assist pop-up

Purpose: Provides AI-generated recommendations for attractions and activities based on the tags.

Key Functions:

- AI chat assistant panel
- List of tags for filtering recommendations type
- Emoji-like feedback button for evaluating AI response

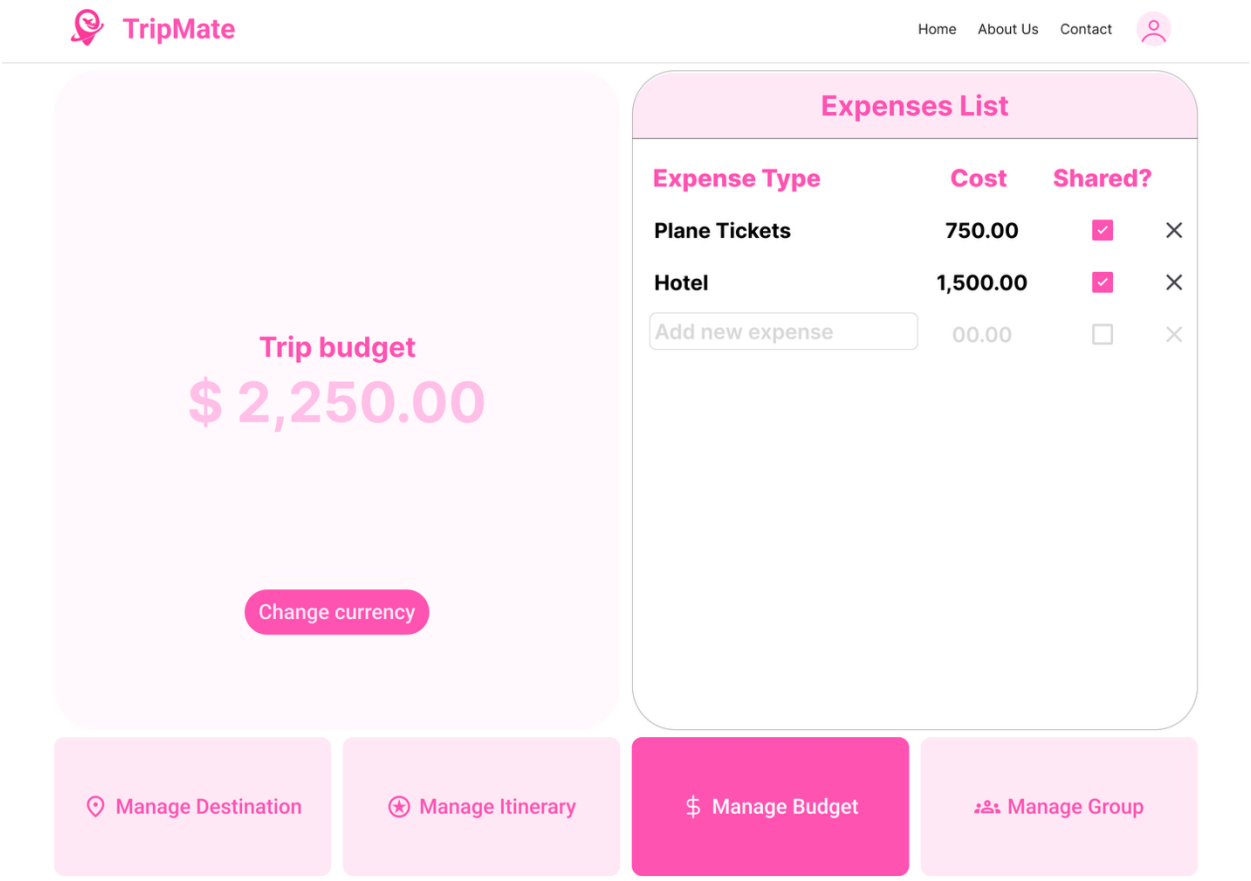


Budget Management page

Purpose: Tracks trip expenses and allows users to manage shared budgets.

Key Functions:

- Expense list with cost breakdowns
- Shared cost checkbox
- Total trip budget summary and currency selection

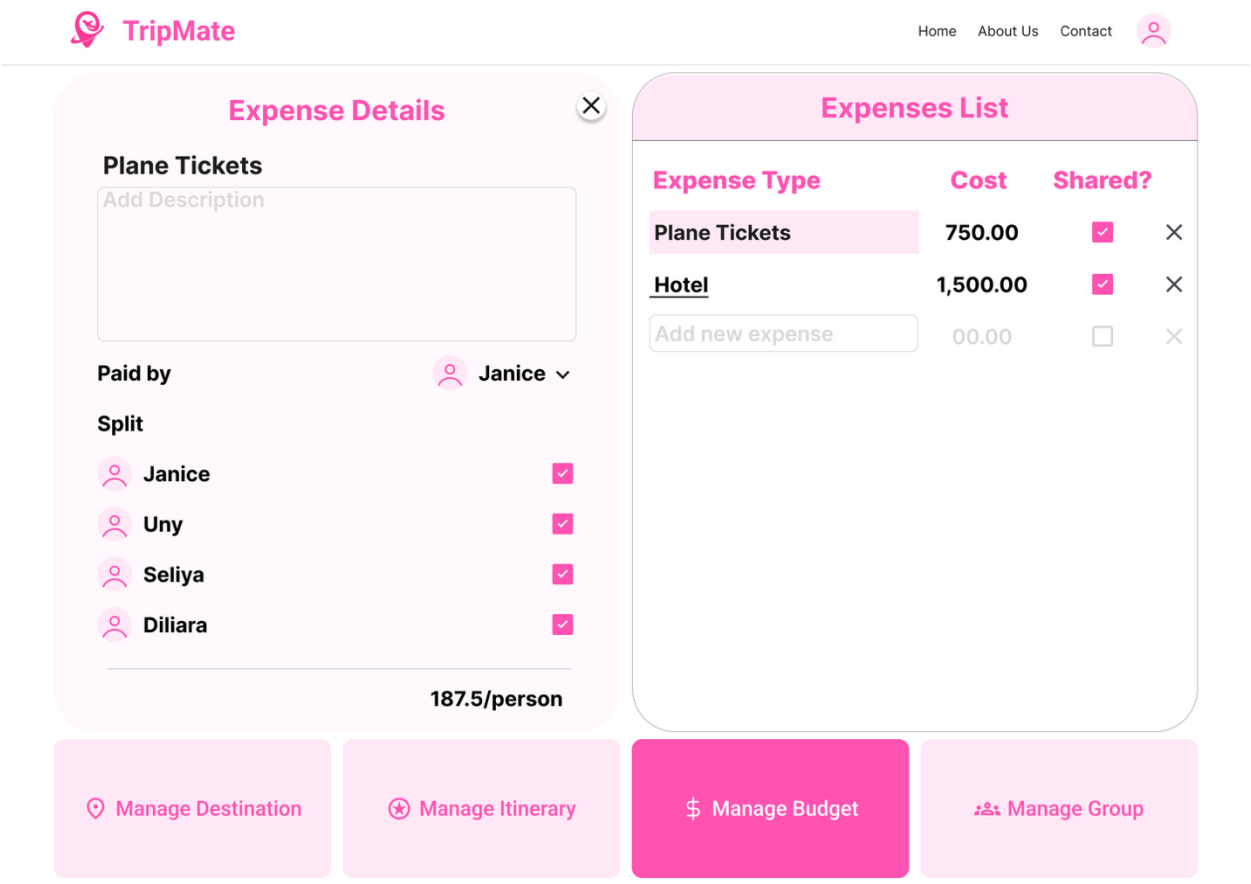


Budget Management page / Shared Expense Details view

Purpose: Pop up appears to make details of selected expense

Key Functions:

- Expense Details Panel: Opens when a specific expense (e.g., Plane Tickets) is selected.
- Add Description Field: Enables users to add notes or details about the expense.
- Paid By Dropdown: Identifies who made the payment within the travel group.
- Split Cost Options: Allows users to select which group members are included in the shared cost calculation.
- Per-Person Calculation: Automatically computes and displays how much each participant owes based on selected members.

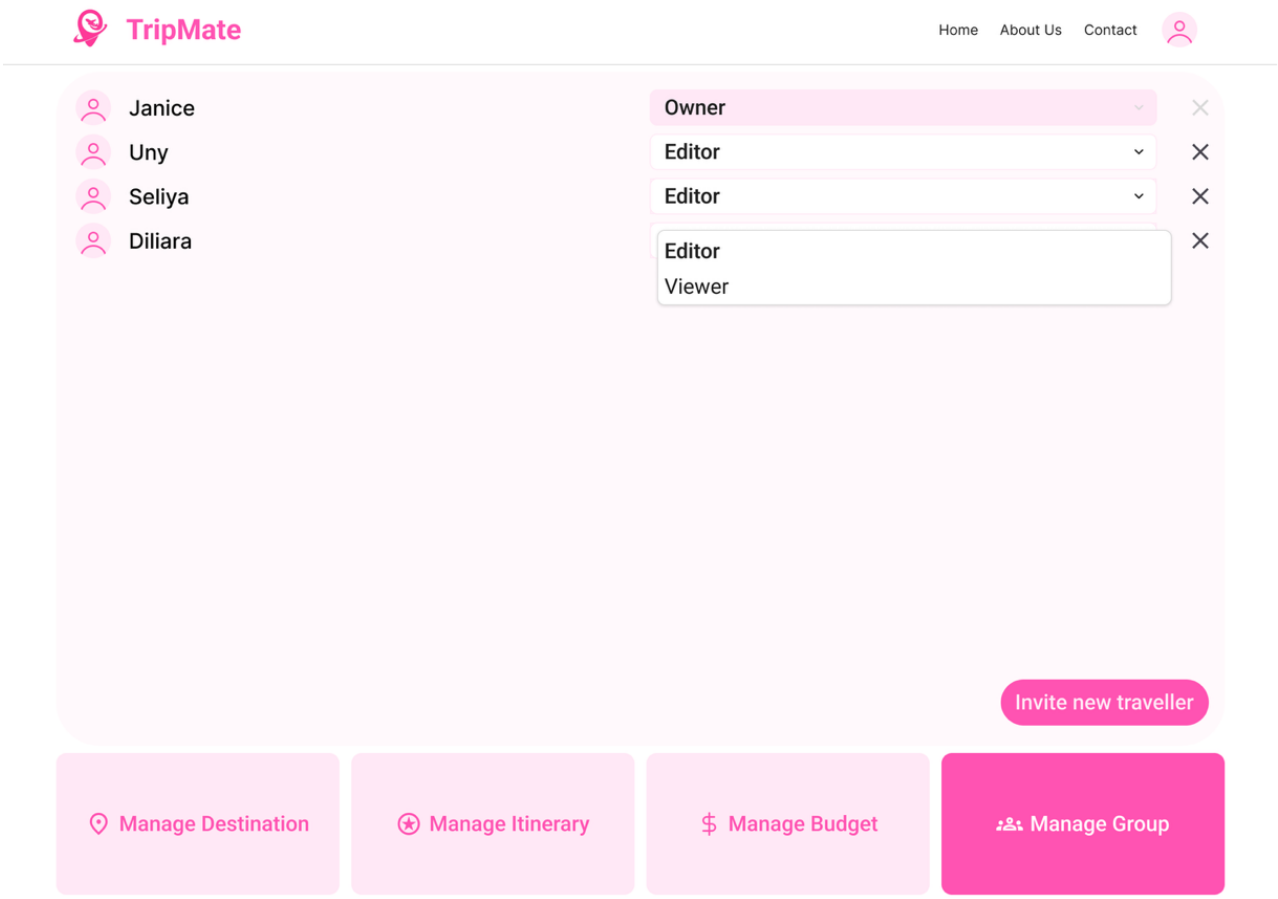


Group Management page

Purpose: Manages team members' roles and collaboration permissions.

Key Functions:

- List of group members
- Role settings (Owner, Editor, Viewer)
- "Invite new traveler" button

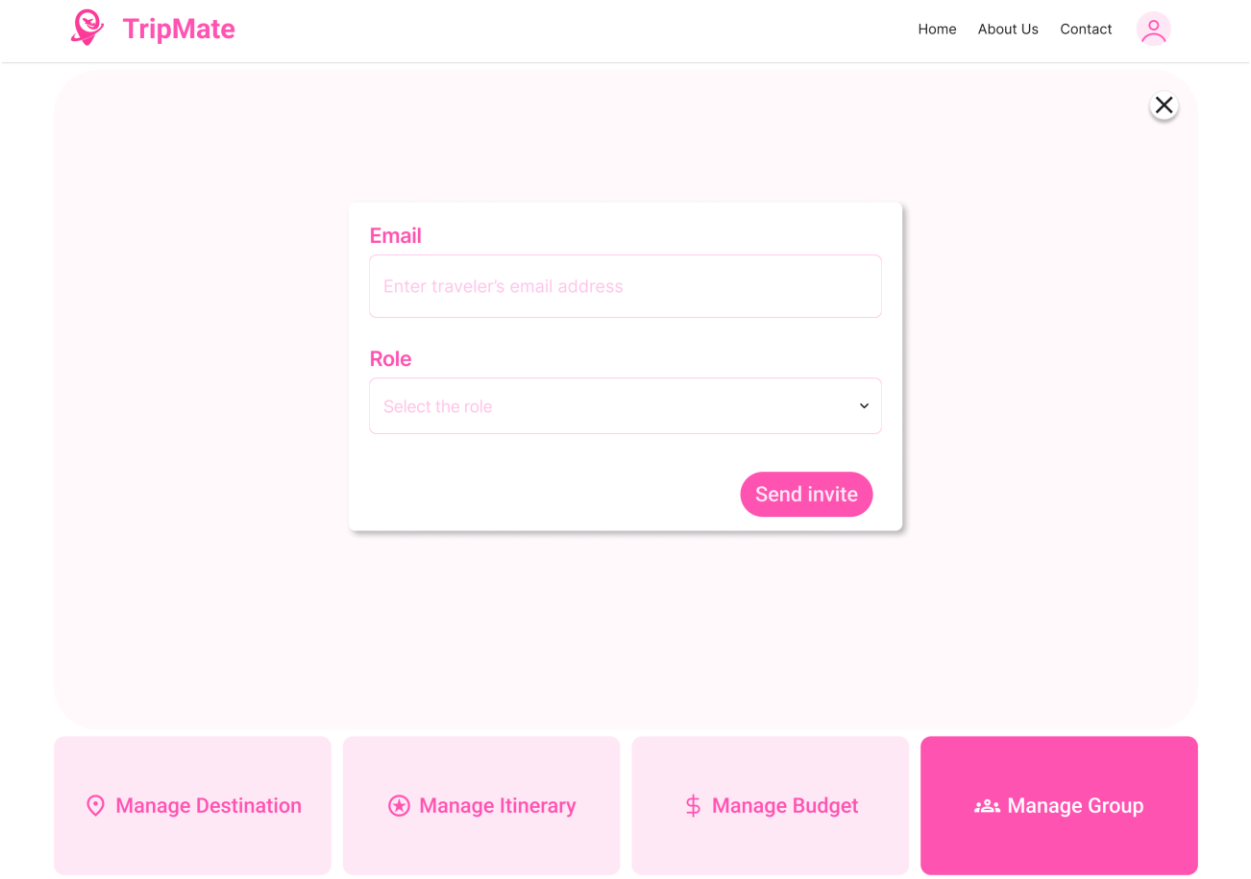


Group Management Page / Invite Traveler Form

Purpose: Allows the Trip Manager to invite new members to join a trip for collaborative planning.

Key Functions:

- Input field for entering traveler's email address
- Dropdown menu to assign user role (Viewer / Editor)
- "Send Invite" button to generate and dispatch invitation link

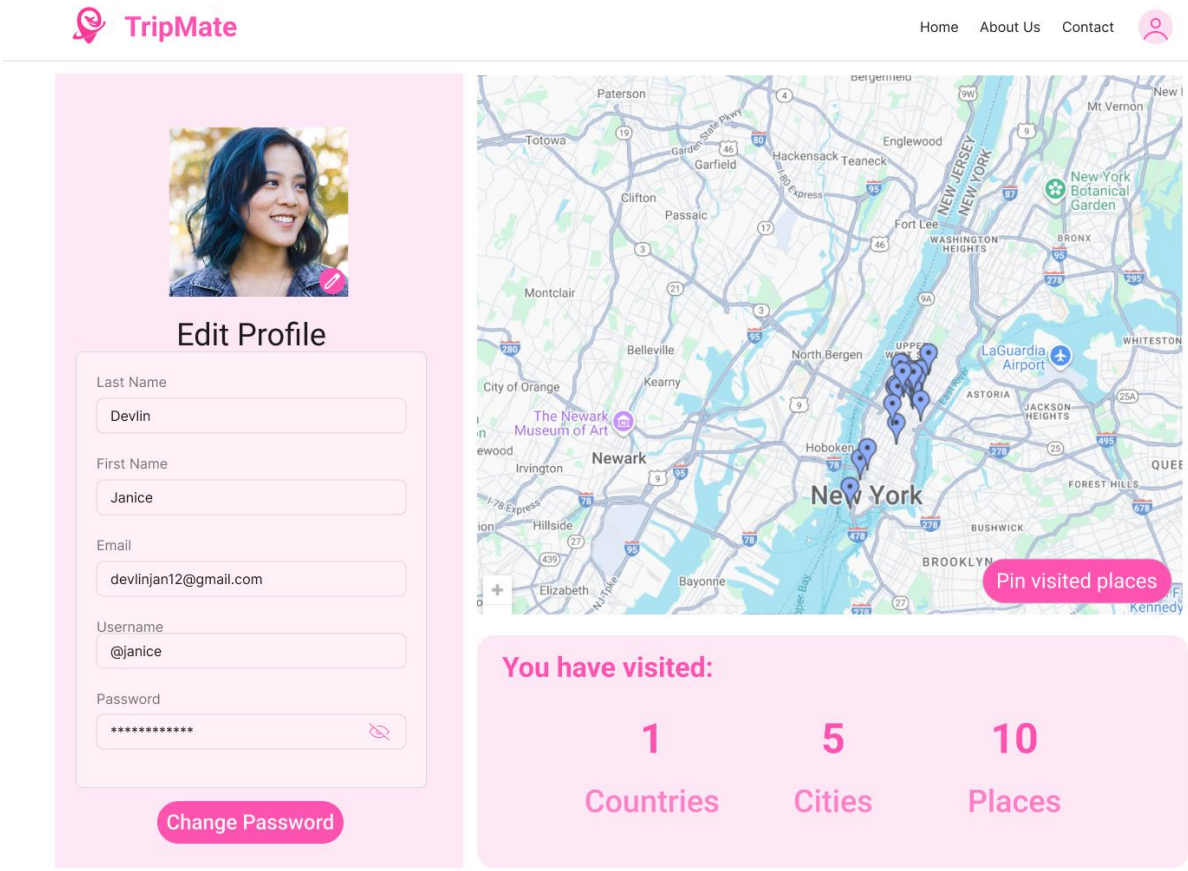


Profile Page

Purpose: Displays the user's personal information and trip history, allowing them to view and manage their travel activities in one place.

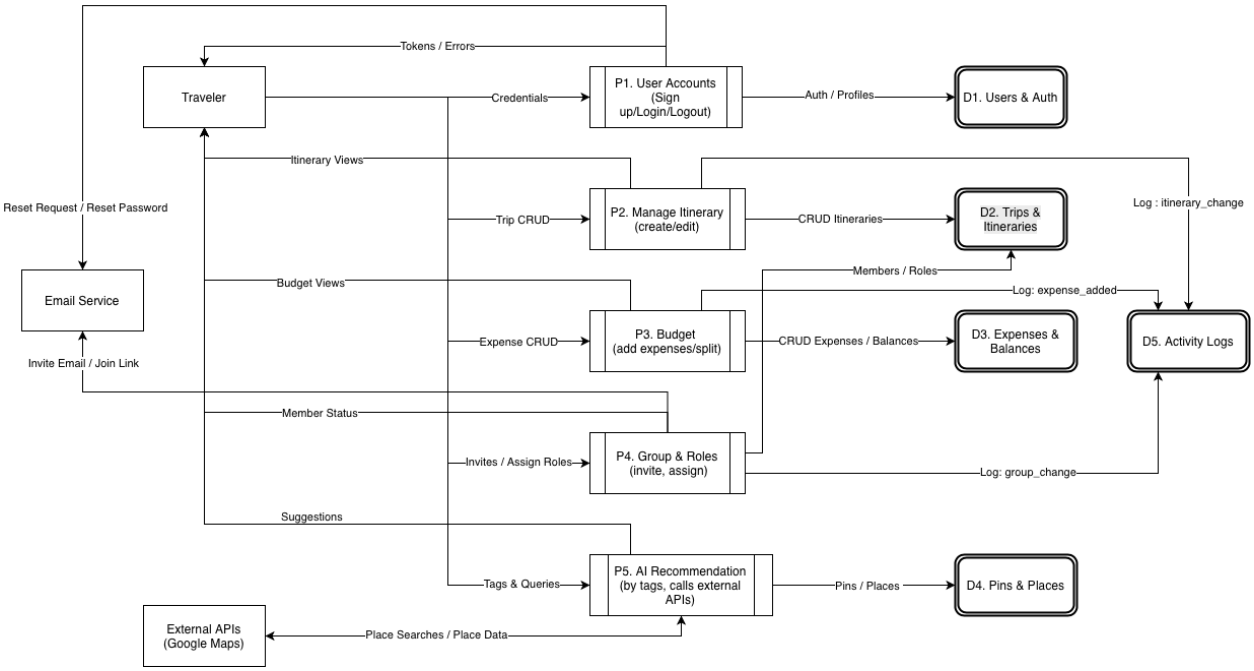
Key Functions:

- Shows user profile details such as avatar, username, and email.
- Displays saved trips with details like destination and travel dates.
- Interactive map highlighting trip locations.
- Options to edit profile, adjust settings, and log out.
- "Add visited place" button for adding pin in the map to mark visited places.



Section 3

3.1 Data Flow Diagrams



3.2 User Stories and related Use Case Scenarios

User Story 1 – Account & Access Management

As a user, I want to register, log in, and manage my account so that I can securely access my personalized trips and preferences.

Use Cases:

- Register an account
- Log in
- Log out
- Recover forgotten password

Section	Description	Description
Use Case ID	UC-1.1	UC-1.2
Use Case Name	Register an Account	Log in
Primary Actor	Guest User	Registered User
Goal	Create a new, secure user account	Access account & Trip dashboard
Preconditions	User not logged in; email not registered in system	Valid registered account exists
Postconditions	New account created; user may log into account and start	User session created

Triggering event	User selects “Register”	User selects “Log In”
Main Success Scenario	1. System opens Registration page 2. User enters required information 3. System validates inputs 4. System creates account 5. System confirms success 6. System redirects to Login page	1. System opens Login page 2. User enters email & password 3. System validates credentials 4. System logs user in 5. System redirects to Home page
Exception Conditions	- Email already exists in system - Invalid password or missing fields	- Invalid credentials
Related Use Cases	- UC-1.2 Log In	- UC-1.1 Register an Account - UC-1.4 Recover Forgotten Password - UC-1.3 Log Out

Section	Description	Description
Use Case ID	UC-1.3	UC-1.4
Use Case Name	Log Out	Recover Forgotten Password
Primary Actor	Logged-in User	Registered User
Goal	End User Session	Regain access to account
Preconditions	User is logged in	Account exists; user not logged in
Postconditions	Session terminated; user returned to public page	Password updated; user can log in
Triggering event	User selects “Log Out”	User selects “Forgot Password”
Main Success Scenario	1. System clears session 2. System redirects to Landing page	1. System opens Password Recovery page 2. User enters email 3.. System sends reset link 4. User opens link 5. User enters new password 6. System resets password 7. System redirects to Login page
Exception Conditions		-Email not found (error) -Password validation fails
Related Use Cases	- UC-1.2 Log In	- UC-1.2 Log In

User Story 2 – Trip Creation & Personal Planning

As a user, I want to create, edit, and manage my trip so that I can plan my itinerary, destinations, and budget in one place.

Use Cases:

- Create new trip

- Edit existing trip
- Edit destinations
- Add daily itinerary items
- Add and delete trip budget
- View trip map with pinned locations

Section	Description	Description
Use Case ID	UC-2. 1	UC-2.2
Use Case Name	Create New Trip	Edit Existing Trip
Primary Actor	Logged-in User	Trip Owner / Editor
Goal	Start a new trip plan	Update core trip information
Preconditions	User authenticated	Trip exists; user has editing rights
Postconditions	Trip created and saved	Trip details updated
Triggering event	User selects “Create Trip”	User opens an existing trip
Main Success Scenario	1. System opens Destination Management page 2. User fills trip details 3. System validates input 4. System saves trip	1. System opens Destination Management page 2. User updates fields 3. System saves updates
Exception Conditions	- Invalid input	- Invalid input - User doesn’t have editing permissions
Related Use Cases	- UC-2.2 Edit Existing Trip - UC-2.3 Edit Destinations - UC-2.4 Add Daily Itinerary Items - UC-2.5 Add/Edit Trip Budget - UC-2.6 View Trip Map	- UC-2.1 Create New Trip - UC-2.3 Edit Destinations - UC-2.4 Add Daily Itinerary Items - UC-2.5 Add/Edit Trip Budget

Section	Description	Description
Use Case ID	UC-2.3	UC-2.4
Use Case Name	Edit Destinations	Add Daily Itinerary Items
Primary Actor	Trip Owner / Editor	Trip Owner / Editor
Goal	Manage trip destinations	Add or edit daily trip activities
Preconditions	Trip exists; user has editing rights	Trip exists; trip dates are set
Postconditions	Trip destination updated	Itinerary updated
Triggering event	User selects “Manage Destination”	User selects “Manage Itinerary”
Main Success Scenario	1. System opens Destination Management page 2. User enters new destination 3. System validates destination 4. System saves changes	1. System opens Itinerary Management page 2. User adds/deletes itinerary items 3. System validates changes 4. System saves changes

Exception Conditions	- Invalid input (non-existent city) - User doesn't have editing permissions	- User doesn't have editing permissions
Related Use Cases	- UC-2.1 Create New Trip - UC-2.2 Edit Existing Trip - UC-2.6 View Trip Map	- UC-2.1 Create New Trip - UC-2.2 Edit Existing Trip - UC-3.3 Edit Shared Itinerary Collaboratively

Section	Description	Description
Use Case ID	UC-2.5	UC-2. 6
Use Case Name	Add / Edit Trip Budget	View Trip Map
Primary Actor	Trip Owner / Editor	Any Trip Member
Goal	Manage financial plan for trip	View map with pinned destinations
Preconditions	Trip exists	Trip exists
Postconditions	Budget updated and recalculated	Map is displayed to user
Triggering event	User selects "Manage Budget"	User selects "Manage Itinerary"
Main Success Scenario	1. System opens Budget Management page 2. User adds/deletes budget items 3. System validates input 4. System saves changes 5. System recalculates totals	1. System opens Itinerary Management page 2. System loads and displays places pins
Exception Conditions	- Invalid costs input - Missing fields - User doesn't have editing permissions	- No places pinned
Related Use Cases	- UC-2.1 Create New Trip - UC-2.2 Edit Existing Trip - UC-3.5 Add/Edit Shared Budget Item - UC-3.6 View Group Trip Budget & Expenses	- UC-2.3 Edit Destinations

User Story 3 – Group & Collaboration Management

As a group traveler or trip manager, I want to manage trip members and share updates so that everyone can collaborate on the same plan.

Use Cases:

- Invite users to join a trip
- Assign roles (viewer/editor)
- Edit shared itinerary collaboratively
- Real-time updates on trip board
- Add or edit shared budget items
- View total group trip budget and expenses

Section	Description	Description
Use Case ID	UC-3.1	UC-3.2
Use Case Name	Invite Users to Join Trip	Assign Roles (Viewer / Editor)
Primary Actor	Trip Owner	Trip Owner
Goal	Add members to collaborate	Control member permissions
Preconditions	Trip exists	Trip exists; group member(s) exist
Postconditions	Invitation sent; member added if invitation accepted	Permissions updated
Triggering event	Admin selects “Invite New Member”	Admin selects “Manage Group”
Main Success Scenario	1. User enters invitee’s email & selects role 2. System sends invite 3. Invitee accepts invitation -> System provides invitee access to trip	1. System opens Group Management page 2. User selects a member and changes their role 3. System updates permissions
Exception Conditions	- User not found - Invitation declined	- Role was not changed - No members found
Related Use Cases	- UC-3.2 Assign Roles - UC-3.3 Edit Shared Itinerary Collaboratively - UC-3.5 Add/Edit Shared Budget Items	- UC-3.1 Invite Users - UC-3.3 Edit Shared Itinerary Collaboratively - UC-3.5 Add/Edit Shared Budget Items

Section	Description	Description
Use Case ID	UC-3.3	UC-3.4
Use Case Name	Edit Shared Itinerary Collaboratively	Real-Time Trip Board Updates
Primary Actor	Trip Owner / Editor	Any Trip Member
Goal	Collaboratively update itinerary	View live updates from others
Preconditions	Trip exists; group member(s) exist; user has editing rights	Trip exists; group member(s) exist; updates happening
Postconditions	Itinerary list updated	Updates displayed to all group members
Triggering event	Editor starts changing itinerary	User is on the trip board and a change occurs
Main Success Scenario	1. Editor adds/deletes items 2. System validates changes 3. System saves updates 4. System synchronizes updates among group members	1. User is viewing the trip board 2. Another member makes a change (e.g., edits itinerary, budget). 3. System automatically pushes the update 4. The user's view refreshes to show the new data
Exception Conditions	- Concurrent edits	- Offline mode or lost connection

Related Use Cases	- UC-2.4 Add Daily Itinerary Items - UC-3.4 Real-Time Trip Board Updates	- UC-3.3 Edit Shared Itinerary - UC-3.5 Add/Edit Shared Budget Items - UC-3.6 View Group Trip Budget & Expenses

Section	Description	Description
Use Case ID	UC-3.5	UC-3.6
Use Case Name	Add / Edit Shared Budget Item	View Group Trip Budget & Expenses
Primary Actor	Trip Owner / Editor	Any Trip Member
Goal	Manage group budget collaboratively	View overall trip budget and list of expenses
Preconditions	Trip exists; group member(s) exist; user has editing rights	Trip exists; group member(s) exist; itinerary list has items
Postconditions	Expenses list updated	Budget viewed
Triggering event	Editor opens Shared Budget Details view	User selects “Manage Budget”
Main Success Scenario	1. System opens Shared Expense Details view 2. Editor edits expense details 3. System validates changes 4. System saves updates 5. System recalculates totals	1. System opens Budget Management page 2. User views total trip budget 3. User views trip expenses and their details
Exception Conditions	- Invalid input	- No expenses found
Related Use Cases	- UC-2.5 Add/Edit Trip Budget - UC-3.6 View/Edit Total Group Budget	- UC-2.5 Add/Edit Trip Budget - UC-3.5 Add/Edit Shared Budget Item

User Story 4 – Smart Recommendations (AI Module)

As a user, I want to receive personalized recommendations for destinations, activities, and experiences so that I can easily discover options that fit my trip.

Use Cases:

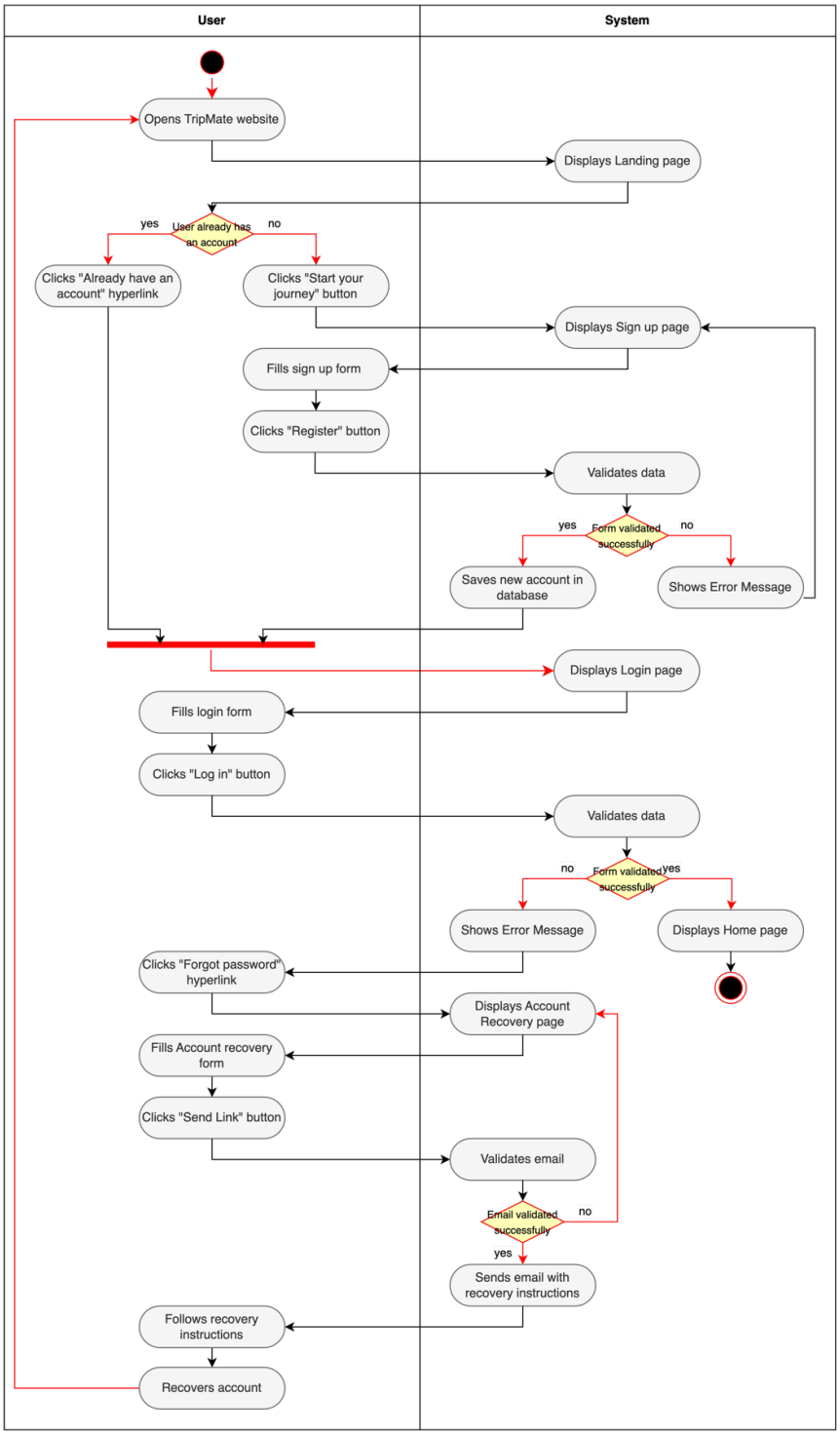
- Get activity or destination suggestions based on tags (e.g., #Food, #Sightseeing, #Adventure)
- Provide feedback on suggestions (useful/not relevant)

Section	Description	Description
Use Case ID	UC-4.1	UC-4.2
Use Case Name	Get AI Activity / Destination Recommendations	Provide Feedback on AI Suggestions
Primary Actor	Trip User	Trip User
Goal	Receive personalized suggestions	Improve future recommendations accuracy

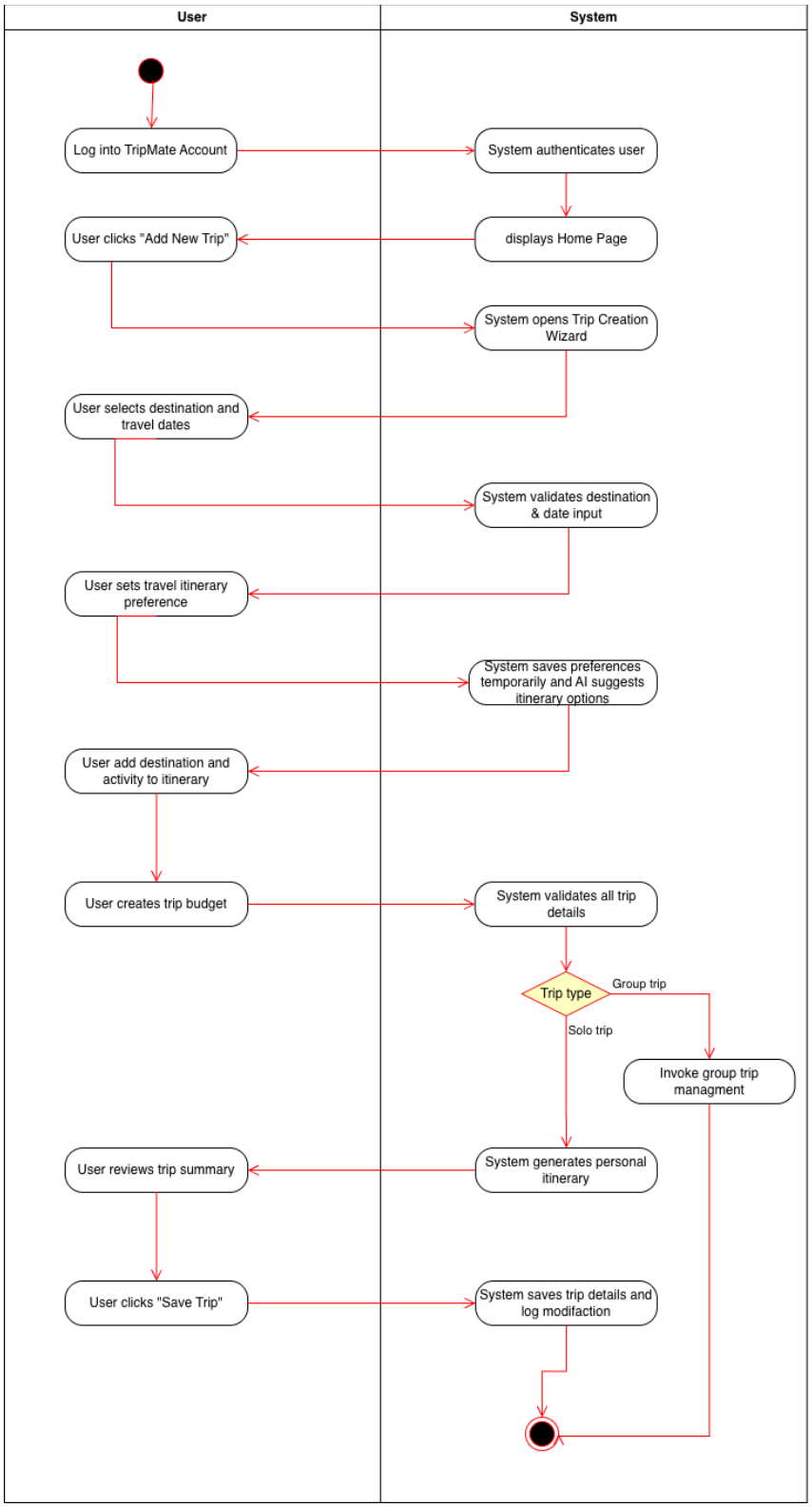
Preconditions	Trip exists; trip dates are set	Recommendations are generated and displayed
Postconditions	Suggestions generated and displayed	Feedback stored; AI model updated
Triggering event	User opens AI assistant	User clicks Thumbs Up/ Thumbs Down
Main Success Scenario	1. System opens AI Assistant view 2. User selects tags 3. User types the request 4. AI generates suggestions	1. System stores feedback 2. System Improves AI model
Exception Conditions	- Tag has not been selected - No suitable results found	- No feedback was provided
Related Use Cases	- UC-4.2 Provide Feedback - UC-2.1 Create Trip (trip data used as input)	- UC-4.1 AI Recommendations

3.3 Activity Diagrams

AD1 Account & Access Management

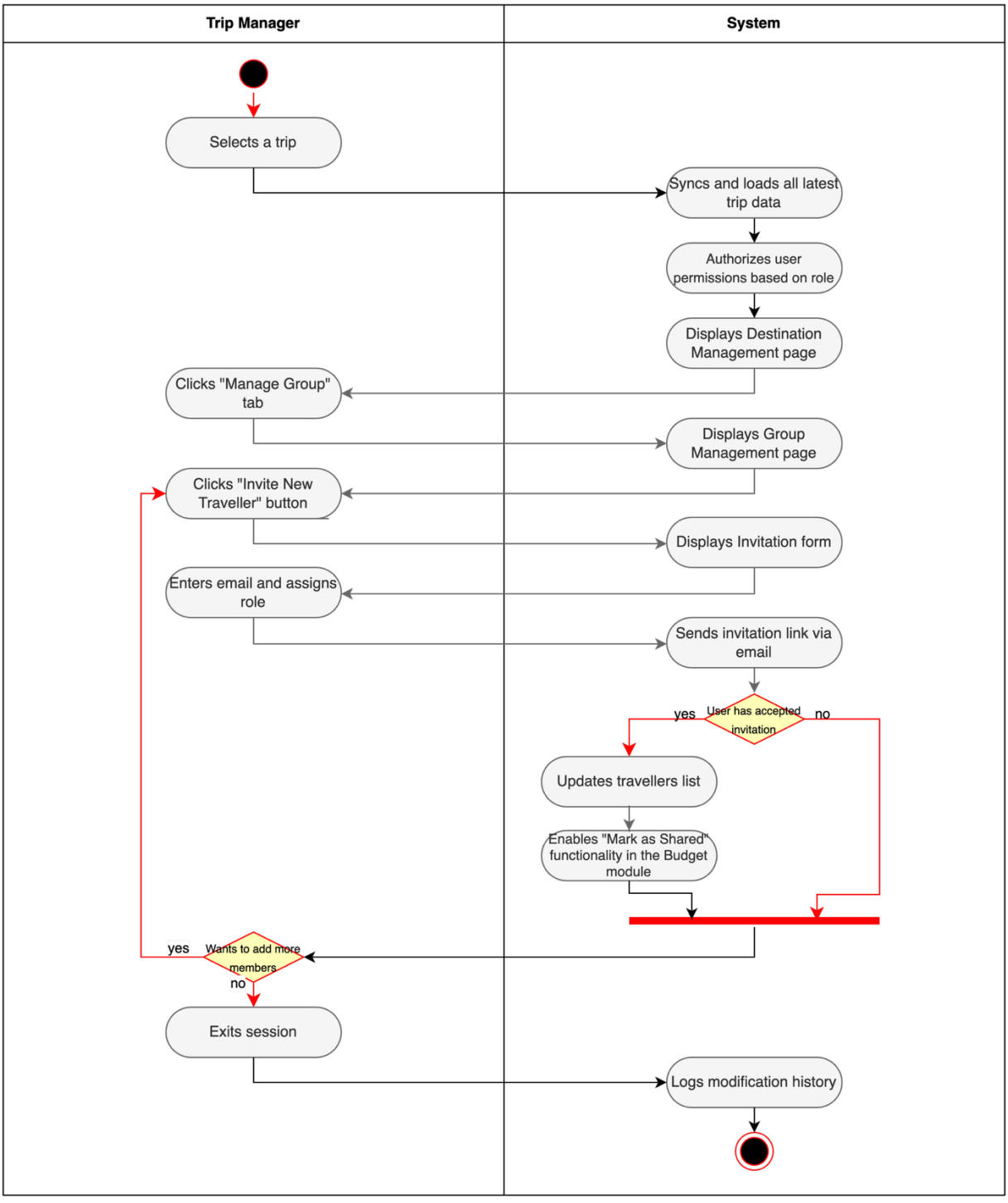


AD2 Trip Creation & Individual Traveler Trip Planning

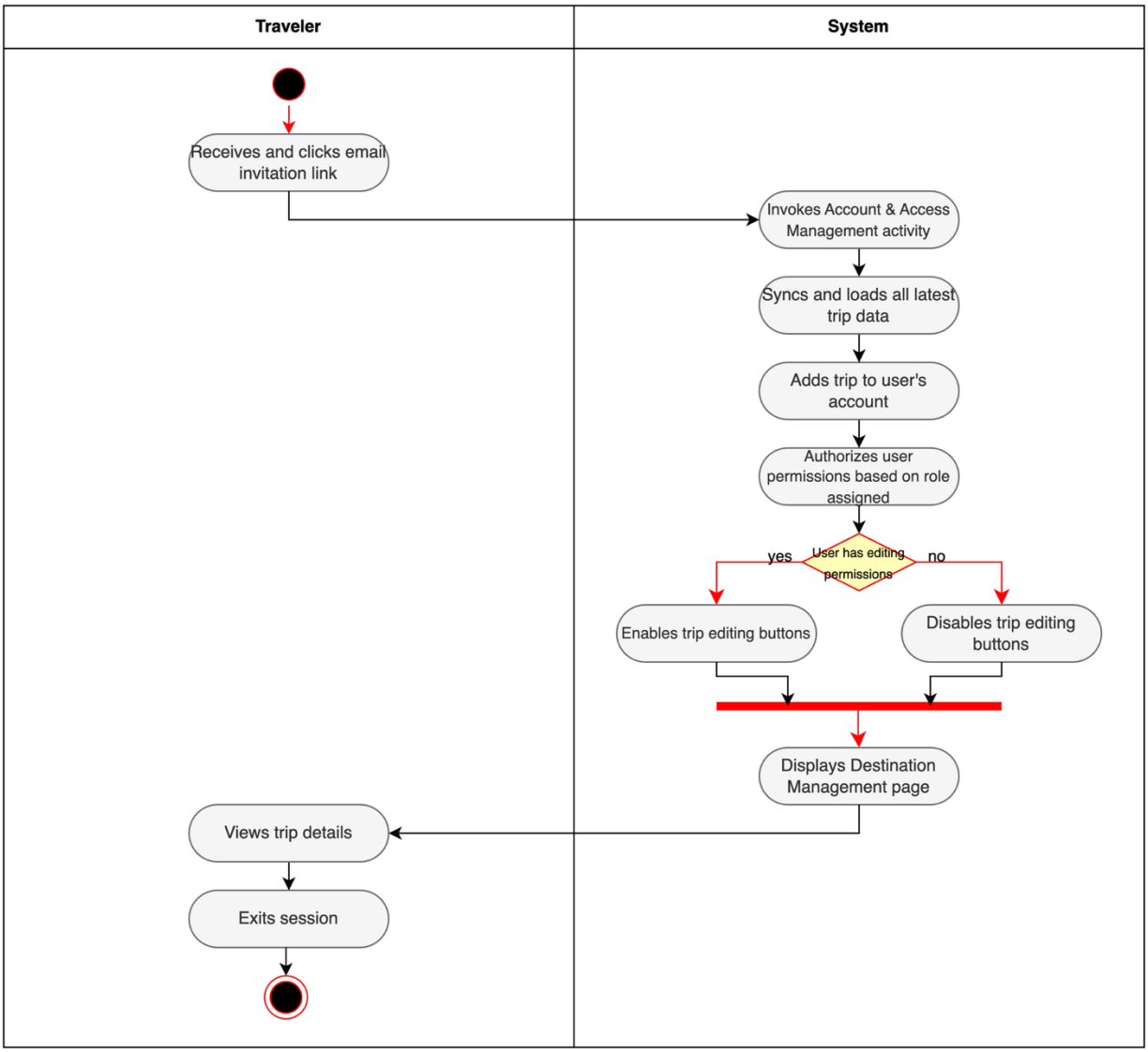


AD3 Group & Collaboration Management

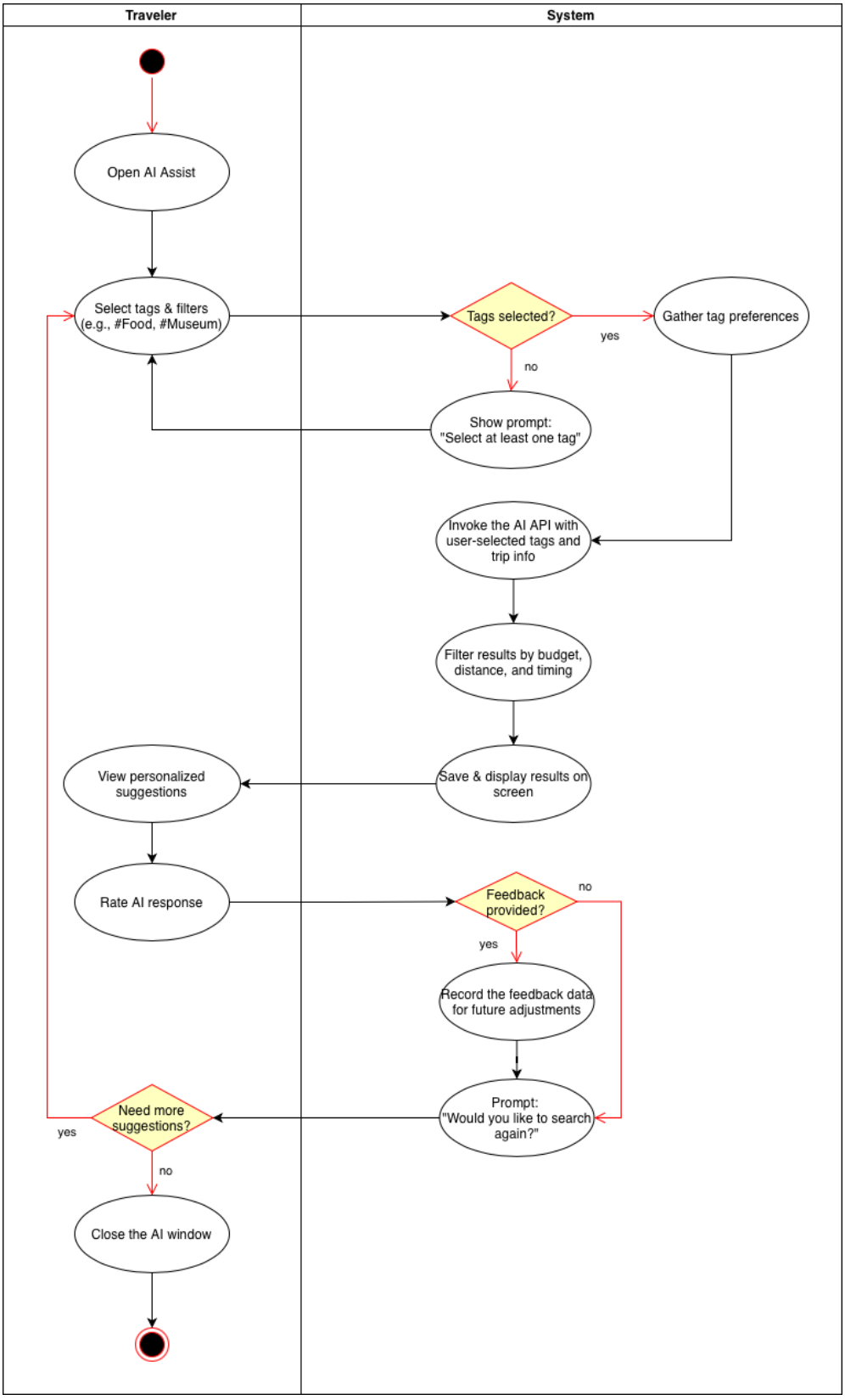
AD3.1 Trip Manager Perspective



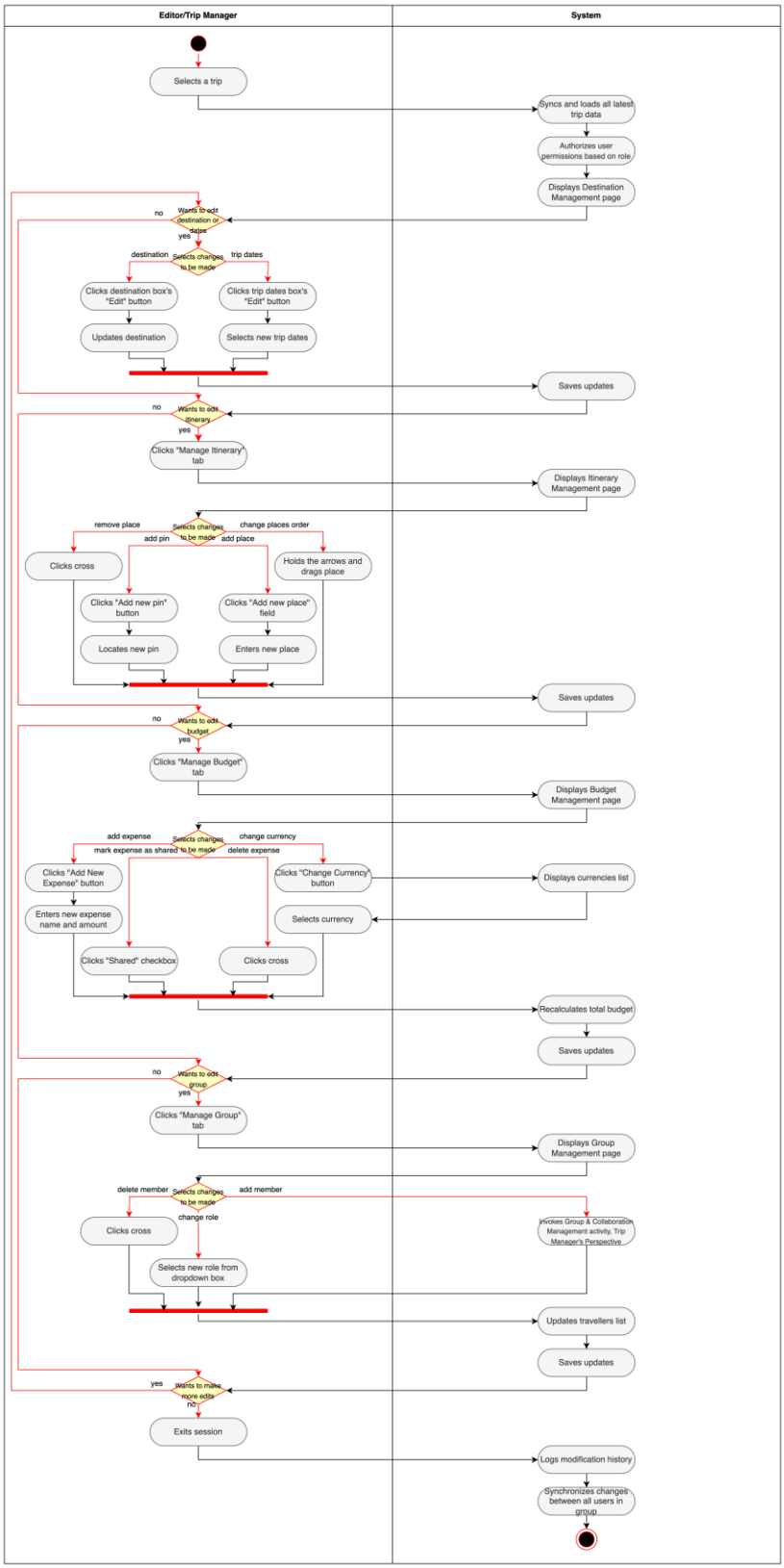
AD3.2 General Traveler Perspective



AD4 Smart Recommendations (AI Module)



AD5 Editing Existing Trip



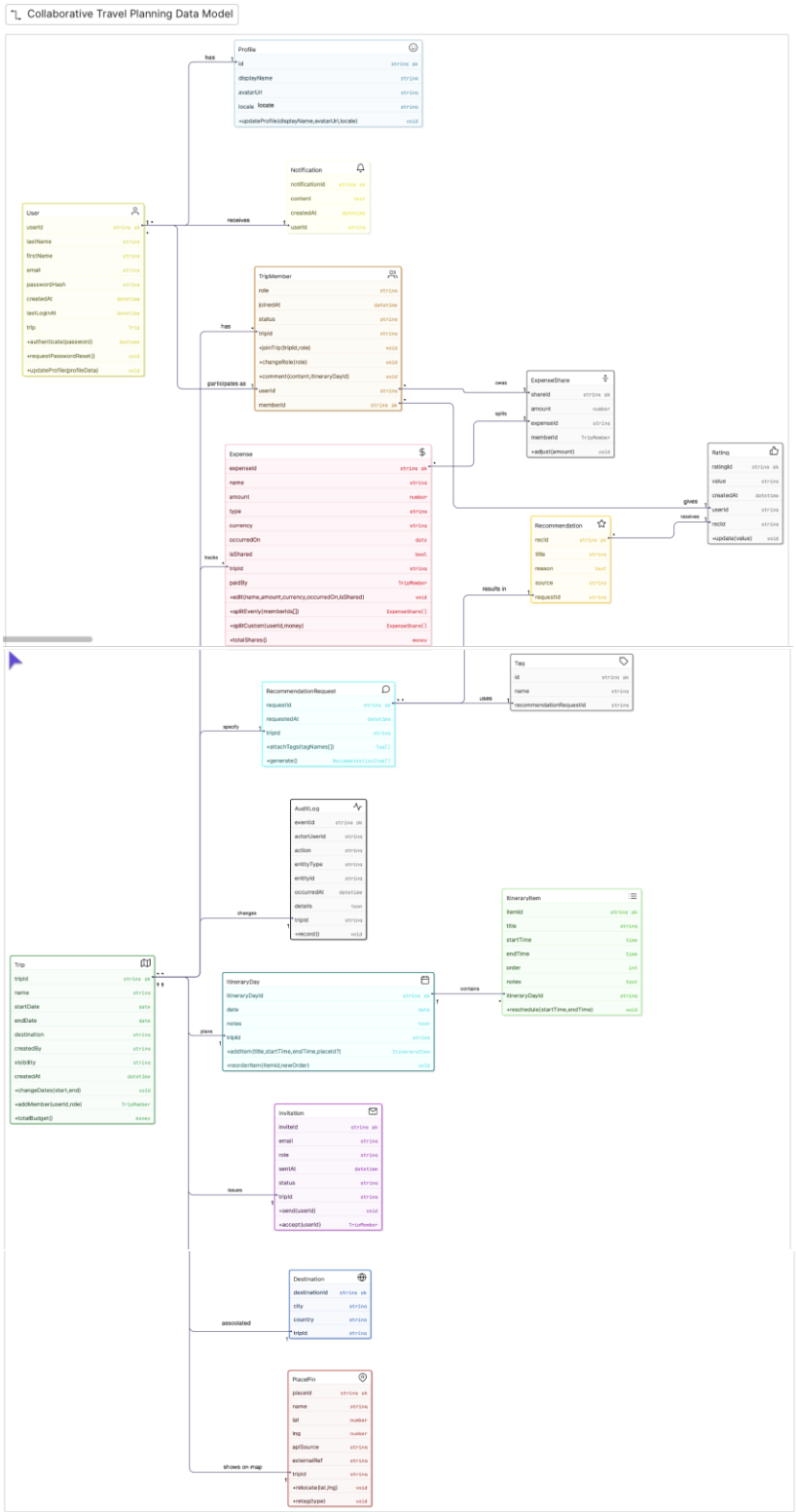
3.4 Business Rules

Business Rule #	Description	Activity Diagram	Related UCS	Applicable Interface
BR1	User must provide unique email during registration.	AD1	UC1	Sign-up Page
BR2	Passwords must meet minimum length (8 chars).	AD1	UC2	Sign-up Page
BR3	Roles (Owner/Editor/Viewer) define permissions.	AD3.1	UC3	Group Management Page / Invite Traveler Page / Backend System
BR4	Expenses must include name + amount > 0.	AD2	UC3	Budget Page
BR5	Users cannot delete shared trips unless they are owner.	AD5	UC4	Home Page
BR6	Expense entries must reflect on-screen within 2 seconds of submission.	AD2, AD5	UC5	Budget Page
BR7	AI suggestions generated only when tags are selected.	AD4	UC6	AI Assist Pop-up
BR8	Users can rate AI suggestions to improve future recommendations.	AD4	UC7	AI Assist Pop-up
BR9	Passwords must be stored using secure hashing and salting techniques.	AD1	UC1/UC2	Backend System
BR10	Users are logged out automatically after 30 minutes of inactivity.	AD1	UC2	Backend System
BR11	Account recovery must be completed via secure email verification.	AD1	UC3	Account Recovery Page
BR12	All trip board edits must synchronize across users within 1 second.	AD5	UC4	Backend System
BR13	Users must have internet access to use TripMate features.	AD2	UC5	Entire Website
BR14	The system must support responsive design for screens ≥ 360px width.	AD2	UC6	All UI pages
BR15	All changes to trip details must be logged for auditing purposes.	AD5	UC4/UC5	Backend System
BR16	The system must restrict access to third-party API credentials.	AD4	UC8	Backend System
BR17	Users must be able to view and manage trip budgets in their local currency.	AD2, AD5	UC5	Budget Page

BR18	The system must comply with GDPR and PIPEDA regulations.	AD1	UC8	Backend System
BR19	Accessibility features must be available for users with disabilities.	AD2	UC9	All UI Pages
BR20	Each itinerary can contain a maximum of 100 scheduled activities.	AD2, AD5	UC4	Itinerary Management Page
BR21	Each trip can include up to 50 pinned map locations.	AD2, AD5	UC6	Itinerary Management Page
BR22	Users must be authenticated before accessing any trip-related features.	AD1	UC2/UC4	Backend System
BR23	Users must be notified of any failed API calls (e.g., map or recommendation).	AD4	UC6/UC7	AI Assist Pop-up / Itinerary Management Page

Section 4 – Domain Class Diagram

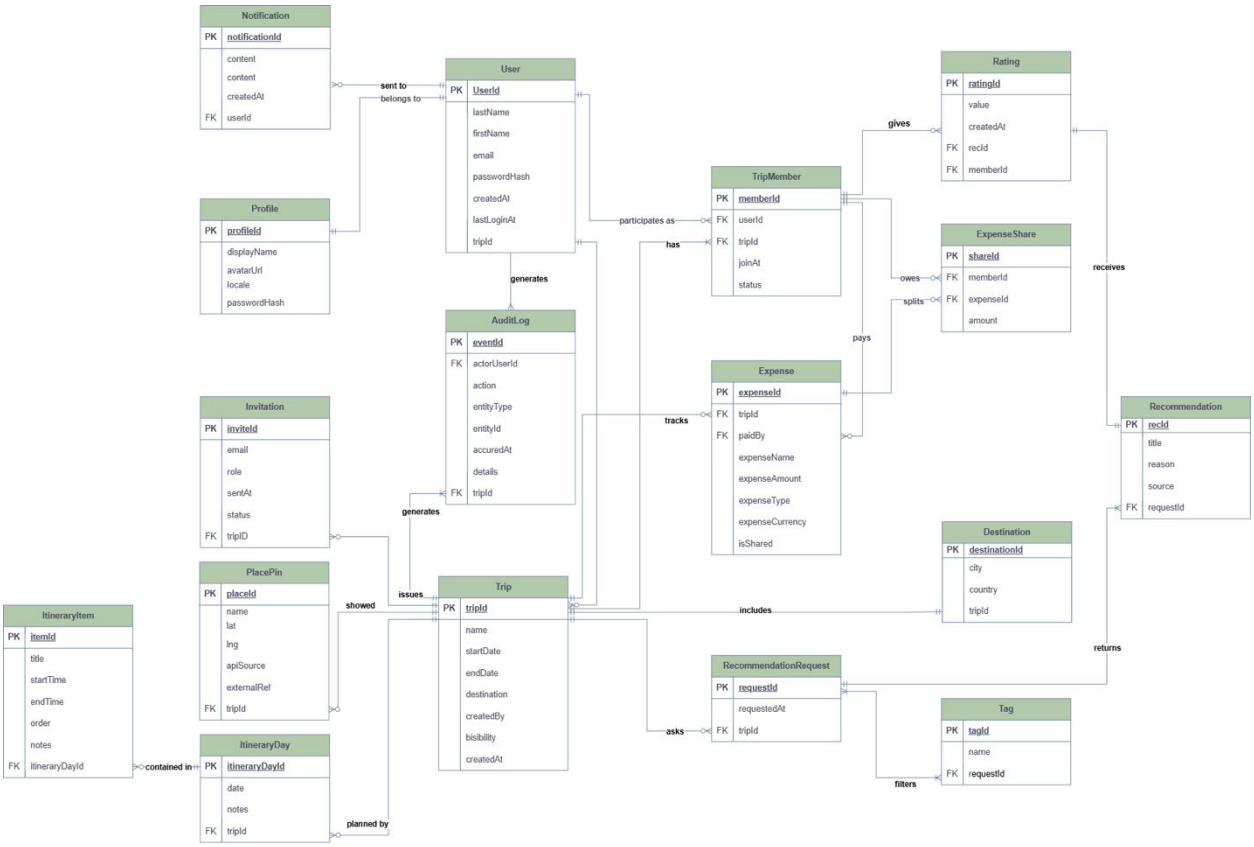
https://app.eraser.io/workspace/ZmsKZR1yf0RWnULkiTS0



Section 5 – Database

5.1 Entity Relationship Diagram

https://app.diagrams.net/#G1RjXLuucy8uk89e2sj_hUOolyj7K7TRUp#%7B%22pageId%22%3A%22ybfmW6ZoMLle0m1rGmb9%22%7D/



5.2 Data Dictionary

5.2.1 User

Field	Data Type	Key	Nullable	Description
userId	VARCHAR(50)	PK	No	Unique identifier for each user.
firstName	VARCHAR(50)		No	User's first name.
lastName	VARCHAR(50)		No	User's last name.
email	VARCHAR(100)	UNIQUE	No	User's email address (used for login).
passwordHash	VARCHAR(255)		No	Hashed password for authentication.
createdAt	DATETIME		No	Timestamp of account creation.
lastLoginAt	DATETIME		Yes	Last login timestamp.

5.2.2 Profile

Field	Data Type	Key	Nullable	Description
profileId	VARCHAR(50)	PK	No	Unique profile identifier.
userId	VARCHAR(50)	FK → Users(userId)	No	Associated user.
displayName	VARCHAR(50)		Yes	Public display name.
avatarUrl	VARCHAR(255)		Yes	Profile image URL.
locale	VARCHAR(10)		Yes	Language/region code (e.g., en-CA).

5.2.3 Trips

Field	Data Type	Key	Nullable	Description
tripId	VARCHAR(50)	PK	No	Unique trip identifier.
name	VARCHAR(100)		No	Trip name/title.
description	TEXT		Yes	Detailed description of the trip.
startDate	DATE		Yes	Trip start date.
endDate	DATE		Yes	Trip end date.
destinationCity	VARCHAR(50)		Yes	Main destination city.
destinationCountry	VARCHAR(50)		Yes	Main destination country.
destinationTimezone	VARCHAR(20)		Yes	Time zone of destination.
createdBy	VARCHAR(50)	FK → Users(userId)	No	User who created the trip.
visibility	VARCHAR(20)		No	‘Private’, ‘Friends’, or ‘Public’.
createdAt	DATETIME		No	When the trip was created.

5.2.4 TripMember

Field	Data Type	Key	Nullable	Description
tripMemberId	VARCHAR(50)	PK	No	Unique membership record ID.
tripId	VARCHAR(50)	FK → Trips(tripId)	No	Linked trip.
userId	VARCHAR(50)	FK → Users(userId)	No	Linked user.
role	VARCHAR(20)		No	Member role (‘Owner’, ‘Editor’, ‘Viewer’).
joinedAt	DATETIME		No	When user joined the trip.
status	VARCHAR(20)		No	Member status (‘Active’, ‘Removed’, etc.).

5.2.5 Invitation

Field	Data Type	Key	Nullable	Description
inviteId	VARCHAR(50)	PK	No	Invitation record ID.
tripId	VARCHAR(50)	FK → Trips(tripId)	No	Trip being shared.
email	VARCHAR(100)		No	Recipient email.
role	VARCHAR(20)		No	Role offered ('Viewer' by default).
sentAt	DATETIME		No	Timestamp when invitation was sent.
acceptedAt	DATETIME		Yes	When invitation was accepted.
status	VARCHAR(20)		No	'Pending', 'Accepted', or 'Declined'.

5.2.6 Destination

Field	Data Type	Key	Nullable	Description
destinationId	VARCHAR(50)	PK	No	Destination identifier.
tripId	VARCHAR(50)	FK → Trips(tripId)	No	Linked trip (1:1).
city	VARCHAR(50)		Yes	City name.
country	VARCHAR(50)		Yes	Country name.
timezone	VARCHAR(20)		Yes	Time zone string (e.g., America/Toronto).
notes	TEXT		Yes	Notes about destination.

5.2.7 PlacePins

Field	Data Type	Key	Nullable	Description
placeId	VARCHAR(50)	PK	No	Unique identifier for pinned place.
tripId	VARCHAR(50)	FK → Trips(tripId)	No	Trip associated.
name	VARCHAR(100)		Yes	Place name.
type	VARCHAR(50)		Yes	Place category (restaurant, museum, etc.).
lat	DECIMAL(9,6)		Yes	Latitude coordinate.
lng	DECIMAL(9,6)		Yes	Longitude coordinate.
apiSource	VARCHAR(50)		Yes	Source of data (e.g., Google Maps, Yelp).
externalRef	VARCHAR(50)		Yes	External reference ID.

5.2.8 ItineraryDays

Field	Data Type	Key	Nullable	Description
itineraryDayId	VARCHAR(50)	PK	No	Unique ID for itinerary day.

tripId	VARCHAR(50)	FK → Trips(tripId)	No	Trip this day belongs to.
date	DATE		Yes	Calendar date of the day.
notes	TEXT		Yes	General notes for the day.

5.2.9 ItineraryItems

Field	Data Type	Key	Nullable	Description
itemId	VARCHAR(50)	PK	No	Itinerary item ID.
itineraryDayId	VARCHAR(50)	FK → ItineraryDays(itineraryDayId)	No	Parent itinerary day.
title	VARCHAR(100)		Yes	Activity title.
startTime	TIME		Yes	Start time.
endTime	TIME		Yes	End time.
order	INT		Yes	Sequence order of the item.
notes	TEXT		Yes	Additional notes.
placeId	VARCHAR(50)	FK → PlacePins(placeId)	Yes	Linked place (optional).

5.2.10 Expense

Field	Data Type	Key	Nullable	Description
expenseId	VARCHAR(50)	PK	No	Expense identifier.
tripId	VARCHAR(50)	FK → Trips(tripId)	No	Associated trip.
name	VARCHAR(100)		No	Expense title (e.g., “Dinner”).
amount	DECIMAL(10,2)		No	Expense amount.
type	VARCHAR(50)		Yes	Expense category (e.g., Food, Transport).
currency	VARCHAR(10)		Yes	Currency code (e.g., USD).
occurredOn	DATE		Yes	Date when expense occurred.
isShared	BOOLEAN		No	Whether expense is shared.
paidBy	VARCHAR(50)	FK → Users(userId)	Yes	Who paid the expense.

5.2.11 ExpenseShares

Field	Data Type	Key	Nullable	Description
shareId	VARCHAR(50)	PK	No	Unique share record.
expenseId	VARCHAR(50)	FK → Expenses(expenseId)	No	Related expense.
userId	VARCHAR(50)	FK → Users(userId)	No	User who owes or shares the expense.
amount	DECIMAL(10,2)		No	Share amount.
settled	BOOLEAN		No	Whether the share is settled.

5.2.12 RecommendationRequests

Field	Data Type	Key	Nullable	Description
requestId	VARCHAR(50)	PK	No	Request identifier.
tripId	VARCHAR(50)	FK → Trips(tripId)	No	Related trip.
userId	VARCHAR(50)	FK → Users(userId)	No	User requesting recommendations.
requestedAt	DATETIME		No	When the request was made.

5.2.13 Recommendations

Field	Data Type	Key	Nullable	Description
recId	VARCHAR(50)	PK	No	Recommendation identifier.
requestId	VARCHAR(50)	FK → RecommendationRequests(requestId)	No	Source request.
placeId	VARCHAR(50)	FK → PlacePins(placeId)	Yes	Related place (optional).
title	VARCHAR(100)		Yes	Recommendation title.
reason	TEXT		Yes	Why it's recommended.
source	VARCHAR(50)		Yes	Source system (AI, user, API).

5.2.14 Ratings

Field	Data Type	Key	Nullable	Description
ratingId	VARCHAR(50)	PK	No	Rating record ID.
userId	VARCHAR(50)	FK → Users(userId)	No	User who rated.
recId	VARCHAR(50)	FK → Recommendations(recId)	No	Rated recommendation.
value	INT		No	Rating value (1–5).
createdAt	DATETIME		No	When rating was submitted.

5.2.15 Notifications

Field	Data Type	Key	Nullable	Description
notificationId	VARCHAR(50)	PK	No	Notification identifier.
userId	VARCHAR(50)	FK → Users(userId)	No	Target user.
type	VARCHAR(50)		Yes	Notification type (invite, update, etc.).
content	TEXT		Yes	Notification message.
createdAt	DATETIME		No	When notification was created.

readAt	DATETIME		Yes	When user read the notification.

5.2.16 AuditLogs

Field	Data Type	Key	Nullable	Description
eventId	VARCHAR(50)	PK	No	Log record ID.
actorUserId	VARCHAR(50)	FK → Users(userId)	Yes	Who performed the action.
action	VARCHAR(100)		No	Action type (e.g., "TripCreated").
entityType	VARCHAR(50)		Yes	Affected entity type (Trip, Expense, etc.).
entityId	VARCHAR(50)		Yes	Affected entity ID.
occurredAt	DATETIME		No	Timestamp of event.
details	JSON		Yes	Additional event metadata.
tripId	VARCHAR(50)	FK → Trips(tripId)	Yes	Related trip, if applicable.

5.3 SQL Tables & Attributes

5.3.1 Users

```
CREATE TABLE Users (
  userId VARCHAR(50) PRIMARY KEY,
  firstName VARCHAR(50) NOT NULL,
  lastName VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  passwordHash VARCHAR(255) NOT NULL,
  createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
  lastLoginAt DATETIME
);
```

Functions (methods in backend):

- authenticate(password) → boolean
- requestPasswordReset() → void
- updateProfile(profileData) → void

5.3.2 Profiles

```
CREATE TABLE Profiles (
```

```
  profileId VARCHAR(50) PRIMARY KEY,
  userId VARCHAR(50) UNIQUE NOT NULL,
  displayName VARCHAR(50),
  avatarUrl VARCHAR(255),
  locale VARCHAR(10),
  FOREIGN KEY(userId) REFERENCES Users(userId) ON DELETE CASCADE
);
```

Functions:

- updateProfile(displayName, avatarUrl, locale) → void

5.3.3 Trips

```
CREATE TABLE Trips (
  tripId VARCHAR(50) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  startDate DATE,
  endDate DATE,
  destinationCity VARCHAR(50),
  destinationCountry VARCHAR(50),
  destinationTimezone VARCHAR(20),
  createdBy VARCHAR(50) NOT NULL,
  visibility VARCHAR(20) DEFAULT 'Private',
  createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(createdBy) REFERENCES Users(userId) ON DELETE CASCADE
);
```

Functions:

- changeDates(start, end) → void

5.3.4 TripMembers

```
CREATE TABLE TripMembers (
  tripMemberId VARCHAR(50) PRIMARY KEY,
  tripId VARCHAR(50) NOT NULL,
  userId VARCHAR(50) NOT NULL,
  role VARCHAR(20) DEFAULT 'Viewer', -- Owner / Editor / Viewer
  joinedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
  status VARCHAR(20) DEFAULT 'Active',
  FOREIGN KEY(tripId) REFERENCES Trips(tripId) ON DELETE CASCADE,
  FOREIGN KEY(userId) REFERENCES Users(userId) ON DELETE CASCADE
);
```

Functions:

- joinTrip(tripId, role) → void
- changeRole(role) → void
- comment(message) → void

5.3.5 Invitations

```
CREATE TABLE Invitations (
  inviteId VARCHAR(50) PRIMARY KEY,
  tripId VARCHAR(50) NOT NULL,
  email VARCHAR(100) NOT NULL,
  role VARCHAR(20) DEFAULT 'Viewer',
  sentAt DATETIME DEFAULT CURRENT_TIMESTAMP,
  acceptedAt DATETIME,
  status VARCHAR(20) DEFAULT 'Pending',
  FOREIGN KEY(tripId) REFERENCES Trips(tripId) ON DELETE CASCADE
);
```

Functions:

- send(userId) → void
- accept(userId) → TripMember

5.3.6 Destinations

```
CREATE TABLE Destinations (
  destinationId VARCHAR(50) PRIMARY KEY,
  tripId VARCHAR(50) UNIQUE NOT NULL,
  city VARCHAR(50),
  country VARCHAR(50),
  timezone VARCHAR(20),
  notes TEXT,
  FOREIGN KEY(tripId) REFERENCES Trips(tripId) ON DELETE CASCADE
);
```

5.3.7 PlacePins

```
CREATE TABLE PlacePins (
  placeId VARCHAR(50) PRIMARY KEY,
  tripId VARCHAR(50) NOT NULL,
  name VARCHAR(100),
  type VARCHAR(50),
  lat DECIMAL(9,6),
  lng DECIMAL(9,6),
  apiSource VARCHAR(50),
  externalRef VARCHAR(50),
  FOREIGN KEY(tripId) REFERENCES Trips(tripId) ON DELETE CASCADE
);
```

5.3.8 ItineraryDays

```
CREATE TABLE ItineraryDays (
  itineraryDayId VARCHAR(50) PRIMARY KEY,
  tripId VARCHAR(50) NOT NULL,
```

```
  date DATE,
  notes TEXT,
  FOREIGN KEY(tripId) REFERENCES Trips(tripId) ON DELETE CASCADE
);
```

5.3.9 ItineraryItems

```
CREATE TABLE ItineraryItems (
  itemId VARCHAR(50) PRIMARY KEY,
  itineraryDayId VARCHAR(50) NOT NULL,
  title VARCHAR(100),
  startTime TIME,
  endTime TIME,
  `order` INT,
  notes TEXT,
  placeId VARCHAR(50),
  FOREIGN KEY(itineraryDayId) REFERENCES ItineraryDays(itineraryDayId) ON DELETE
CASCADE,
  FOREIGN KEY(placeId) REFERENCES PlacePins(placeId)
);
```

5.3.10 Expenses

```
CREATE TABLE Expenses (
  expenseId VARCHAR(50) PRIMARY KEY,
  tripId VARCHAR(50) NOT NULL,
  name VARCHAR(100),
  amount DECIMAL(10,2),
  type VARCHAR(50),
  currency VARCHAR(10),
  occurredOn DATE,
```

```
    isShared BOOLEAN DEFAULT FALSE,
    paidBy VARCHAR(50),
    FOREIGN KEY(tripId) REFERENCES Trips(tripId) ON DELETE CASCADE,
    FOREIGN KEY(paidBy) REFERENCES Users(userId)
);
```

5.3.11 ExpenseShares

```
CREATE TABLE ExpenseShares (
    shareId VARCHAR(50) PRIMARY KEY,
    expenseId VARCHAR(50) NOT NULL,
    userId VARCHAR(50) NOT NULL,
    amount DECIMAL(10,2),
    settled BOOLEAN DEFAULT FALSE,
    FOREIGN KEY(expenseId) REFERENCES Expenses(expenseId) ON DELETE CASCADE,
    FOREIGN KEY(userId) REFERENCES Users(userId)
);
```

5.3.12 RecommendationRequests

```
CREATE TABLE RecommendationRequests (
    requestId VARCHAR(50) PRIMARY KEY,
    tripId VARCHAR(50) NOT NULL,
    userId VARCHAR(50) NOT NULL,
    requestedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(tripId) REFERENCES Trips(tripId) ON DELETE CASCADE,
    FOREIGN KEY(userId) REFERENCES Users(userId)
);
```

5.3.13 Recommendations

```
CREATE TABLE Recommendations (
```

```
  recId VARCHAR(50) PRIMARY KEY,
  requestId VARCHAR(50) NOT NULL,
  placeId VARCHAR(50),
  title VARCHAR(100),
  reason TEXT,
  source VARCHAR(50),
  FOREIGN KEY(requestId) REFERENCES RecommendationRequests(requestId),
  FOREIGN KEY(placeId) REFERENCES PlacePins(placeId)
);
```

5.3.14 Ratings

```
CREATE TABLE Ratings (
  ratingId VARCHAR(50) PRIMARY KEY,
  userId VARCHAR(50) NOT NULL,
  recId VARCHAR(50) NOT NULL,
  value INT CHECK(value BETWEEN 1 AND 5),
  createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(userId) REFERENCES Users(userId),
  FOREIGN KEY(recId) REFERENCES Recommendations(recId)
);
```

5.3.15 Notifications

```
CREATE TABLE Notifications (
  notificationId VARCHAR(50) PRIMARY KEY,
  userId VARCHAR(50) NOT NULL,
  type VARCHAR(50),
  content TEXT,
  createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
  readAt DATETIME,
```

```
  FOREIGN KEY(userId) REFERENCES Users(userId)
);
```

5.3.16 AuditLogs

```
CREATE TABLE AuditLogs (
  eventId VARCHAR(50) PRIMARY KEY,
  actorUserId VARCHAR(50),
  action VARCHAR(100),
  entityType VARCHAR(50),
  entityId VARCHAR(50),
  occurredAt DATETIME DEFAULT CURRENT_TIMESTAMP,
  details JSON,
  tripId VARCHAR(50),
  FOREIGN KEY(actorUserId) REFERENCES Users(userId),
  FOREIGN KEY(tripId) REFERENCES Trips(tripId)
);
```

5.4 Relationships Summary

Parent Table	Child Table	Type
Users	Profiles	1:1
Users	TripMembers	1: N
Trips	TripMembers	1: N
Trips	Destinations	1:1
Trips	PlacePins	1: N
Trips	ItineraryDays	1: N
ItineraryDays	ItineraryItems	1: N
PlacePins	ItineraryItems	1: N
Trips	Expenses	1: N
Users	Expenses (paidBy)	1: N

Expenses	ExpenseShares	1: N
Trips	RecommendationRequests	1: N
RecommendationRequests	Recommendations	1: N
PlacePins	Recommendations	1: N
Users	Ratings	1: N
Recommendations	Ratings	1: N
Users	Notifications	1: N
Users	AuditLogs	1: N

Section 6 – Project Management

6.1 Work Breakdown Structure

Level 1: Product Goal

- TripMate Web Application

Level 2: Major Deliverables (High-Level Features/Epics)

1. User Authentication & Profile Management
2. Trip & Itinerary Management
3. Budget & Expense Tracking
4. Interactive Trip Map & Places Pinning
5. Group Collaboration
6. AI-Powered Recommendations
7. Personal Places Tracking & Statistics
8. Cross-Platform Usability
9. Deployment, Security & DevOps

Level 3: Iteration/Sprint-Level Breakdown (User Stories/Features)

1. User Authentication & Profile Management

1.1. As a new user, I can register for an account with my email and a secure password so that I can access TripMate.
1.2. As a registered user, I can log in to my account so that I can access my trips and data.
1.3. As a user, I can log out of my account so that I can keep my information secure on shared devices.
1.4. As a user who forgot my password, I can request a password reset via email so that I can regain access to my account.
1.5. As a user, I can view and edit my profile information (display name, avatar) so that I can personalize my account.

2. Trip & Itinerary Management

2.1. As a user, I can create a new trip by providing a destination and dates so that I can start planning.
2.2. As a user, I can view all my trips on a home page so that I can quickly access them.
2.3. As a user, I can edit the details of an existing trip so that I can adjust my plans.
2.4. As a user, I can add, edit, or delete daily activities in an itinerary.
2.5. As a user, I can manually reorder itinerary items via drag-and-drop so that I can sequence my day logically.
2.6. As a user, I can delete a trip so that I can remove plans I no longer need.

3. Budget & Expense Tracking

3.1. As a user, I can add an expense to a trip (name, amount) so that I can track my spending.
3.2. As a user, I can view the total trip expenses budget so that I can monitor my finances.

3.3. As a group traveler, I can mark an expense as "shared" so that it can be split among participants.
3.4. As a group traveler, I can select which user paid an expense so that the payer is correctly recorded for splitting.
3.5. As a group traveler, I can select which group members to split the cost with so that the burden is distributed fairly.
3.6. As a group traveler, I can add notes to shared expenses so that I can record specific details.

4. Interactive Trip Map & Places Pinning

4.1. As a user, I can view an interactive map centered on my trip destination.
4.2. As a user, I can pin locations (restaurants, attractions) to the trip map so that I can visualize my plans.
4.3. As a user, I can see all my pinned locations displayed on the map at once so that I can get a geographical overview of my trip.

5. Group Collaboration

5.1. As a Trip Manager, I can invite users to a trip via their email address so that they can collaborate with me.
5.2. As a Trip Manager, I can assign a role (Editor or Viewer) to a user when inviting them so that I can control their permissions.
5.3. As a Trip Manager, I can remove a user from a trip so that I can manage the group.
5.4. As a collaborator with Editor access, I can edit the details of an existing trip so that I can adjust shared plans.
5.5. As any collaborator, I can see real-time updates made by others on the trip board so that we are always in sync.

6. AI-Powered Recommendations

6.1. As a user, I can request recommendations for places/activities based on tags (e.g., #food, #sightseeing) so that I can discover new ideas.
6.2. As a user, I can view a list of AI-generated recommendations so that I can add them to my itinerary.
6.3. As a user, I can provide positive or negative feedback on a recommendation so that the AI can learn and improve.

7. Personal Places Tracking & Statistics

7.1. As a user, I can see Personal Traveling Statistics section on my profile showing my total number of visited countries, cities, and places so that I can track my travel achievements.
7.2. As a user, I can view a world map on my profile page where all my visited places are pinned so that I can visually relive my travels.
7.3. As a user, I can manually add a visited place to my profile map to include travels not planned in TripMate.
7.4. As a user, when I create a new trip, all its pinned itinerary locations are automatically added to my personal travel history map and stats.

8. Cross-Platform Usability

8.1. As a mobile user, I can use all core TripMate features on my phone's browser so that I can plan on the go.

8.2. As a user, I can experience a consistent and intuitive interface across Chrome, Firefox, and Safari.
8.3. As a user, I can navigate through the web application using the designed pages (Landing, Home, Profile, Destination, Itinerary, Budget, Group).

9. Deployment, Security & DevOps

9.1. As a developer, I can deploy the web application to a cloud platform (e.g., AWS) so that it is publicly accessible.
9.2. As a developer, I have a version-controlled codebase (GitHub) and a defined process for integrating code.
9.3. As a system, I must hash and salt user passwords before storing them in the database so that data is secure.
9.4. As a system, I must encrypt sensitive data in transit and at rest to protect user privacy.
9.5. As a system, I must log users out after 30 minutes of inactivity to prevent unauthorized access.

Level 4: Sprint Tasks (Granular WBS)

1. User Authentication & Profile Management

1.1. User Registration
1.1.1. Implement registration frontend form (React/Vue)
1.1.2. Setup backend API route for user registration
1.1.3. Integrate password hashing (bcrypt)
1.1.4. Connect registration API to database (insert user record)
1.1.5. Unit test registration module
1.1.6. Sprint review & feedback integration

1.2. Login / Logout
1.2.1. Implement login form (frontend)
1.2.2. Create backend login API endpoint
1.2.3. Add login token storage (cookie/local Storage)
1.2.4. Build logout button + session clear logic
1.2.5. Add login error handling
1.2.6. Unit test login/logout functionality
1.2.7. Sprint demo & refinement

1.3. Password Reset
1.3.1. Implement frontend form for email input
1.3.2. Create backend API for password reset requests
1.3.3. Integrate email service to send reset links
1.3.4. Build UI for entering new password
1.3.5. Implement backend logic to validate reset token and update password
1.3.6. Unit test password reset flow
1.3.7. Sprint review

1.4. Profile Management
1.4.1. Implement frontend components
1.4.2. Create backend API for updating profile
1.4.3. Database update queries for user profile fields
1.4.4. Implement file upload for profile picture

1.4.5. Add client-side validation
1.4.6. Unit test profile update flow
1.4.7. Sprint review

2. Trip & Itinerary Management

2.1. Create & View Trips
2.1.1. Implement new trip frontend
2.1.2. Connect trip records to user ID in database
2.1.3. Implement frontend to display list of user's trips
2.1.4. Build backend API to fetch user's trips
2.1.5. Validate date ranges & required fields
2.1.6. Unit test trip creation and listing
2.1.7. Demo & review

2.2. Edit & Delete Trips
2.2.1. Implement edit trip functionality
2.2.2. Build backend API for updating trip details
2.2.3. Implement trip deletion UI and confirmation
2.2.4. Build backend API for deleting trips
2.2.5. Unit test trip editing and deletion
2.2.6. Sprint review

2.3. Daily Itinerary Management
2.3.1. Implement “Add/Edit itinerary item” modal
2.3.2. Backend API for itinerary items (CRUD)
2.3.3. Connect items to specific trip dates
2.3.4. Unit test itinerary API
2.3.5. Sprint review

2.4. Itinerary Drag-and-Drop
2.4.1. Implement drag-and-drop library integration
2.4.2. Add visual feedback for dragging and dropping items
2.4.3. Build backend API to update item order in database
2.4.4. Connect frontend drag events to order-update API
2.4.5. Unit test reordering functionality
2.4.6. Sprint demo

3. Budget & Expense Tracking

3.1. Personal Expense Tracking
3.1.1. Implement add/delete expense UI
3.1.2. Create backend API for expense items (CRUD)
3.1.3. Implement auto-recalculate totals backend logic
3.1.4. Unit test budget module
3.1.5. Sprint demo

3.2. Group Expense Management
3.2.1. Extend expense UI to include "shared" toggle
3.2.2. Add "paid by" user dropdown selector to expense form
3.2.3. Add "split with" multi-select field for group members
3.2.4. Implement cost-sharing backend logic (split equally)

3.2.5. Implement backend logic to add notes to shared expenses
3.2.6. Unit test shared expense logic
3.2.7. Sprint review

4. Interactive Trip Map & Places Pinning

4.1. Map Integration
4.1.1. Choose map provider (Google Maps, Leaflet, Mapbox)
4.1.2. Implement frontend map component
4.1.3. Implement logic to center map on trip destination
4.1.4. Unit test map rendering
4.1.5. Sprint review

4.2. Places Pinning
4.2.1. Implement “Add pin” feature on map
4.2.2. Capture coordinates on map click
4.2.3. Save pin to database linked to trip
4.2.4. Implement logic to fetch and display all pinned locations on map load
4.2.5. Unit test place pinning
4.2.6. Sprint review

5. Group Collaboration

5.1. Member Management
5.1.1. Build “Invite user” UI form (email input, role dropdown)
5.1.2. Backend API for sending invites and storing role
5.1.3. Implement UI to display trip members and their roles
5.1.4. Build "Remove user" functionality UI and backend API
5.1.5. Update backend permission checks for Editors vs. Viewers
5.1.6. Unit test role-based access control
5.1.7. Sprint review

5.2. Real-Time Updates
5.2.1. Integrate WebSocket (Socket.io)
5.2.2. Implement backend logic to broadcast trip changes (itinerary, budget)
5.2.3. Implement frontend logic to listen for and display real-time updates
5.2.4. Unit test real-time behavior
5.2.5. Sprint demo & fixes

6. AI-Powered Recommendations

6.1. AI Recommendations Interface
6.1.1. Create UI for selecting tags (#food, #sightseeing) and requesting recommendations
6.1.2. Implement AI recommendation API call from frontend
6.1.3. Format and display AI suggestion in the UI
6.1.4. Unit test AI response handling
6.1.5. Review & refine

6.2. Feedback System
6.2.1. Add “Thumbs Up / Thumbs Down” buttons to recommendations
6.2.2. Backend endpoint to log user feedback
6.2.3. Store user preferences in database

6.2.4. Unit test feedback logic
6.2.5. Sprint review

7. Personal Places Tracking & Statistics

7.1. Statistics & Profile Map
7.1.1. Design Personal Travel Statistics UI (counts of countries, cities, places visited)
7.1.2. Implement backend logic to calculate statistics from travel history
7.1.3. Design world map component for profile page
7.1.4. Implement backend API to fetch user's visited places for the map
7.1.5. Render visited places as pins on the profile world map
7.1.6. Unit testing
7.1.7. Sprint review

7.2. Travel History Logic
7.2.1. Implement backend logic to auto-add pinned itinerary locations to travel history upon trip creation/completion
7.2.2. Build UI and backend API for manually adding a visited place pin
7.2.3. Unit test history tracking logic
7.2.4. Sprint demo

8. Cross-Platform Usability

8.1. Responsive Web Design
8.1.1. Build mobile-responsive layout grid for all pages
8.1.2. Implement responsive navigation and components
8.1.3. Verify and fix desktop/tablet/mobile compatibility
8.1.4. Test across Chrome, Firefox, and Safari
8.1.5. Sprint review

9. Deployment, Security & DevOps

9.1. Backend Deployment
9.1.1. Setup cloud hosting environment (AWS EC2, Azure App Service, etc.)
9.1.2. Configure production database (e.g., AWS RDS, MongoDB Atlas)
9.1.3. Deploy backend API to cloud server
9.1.4. Configure environment variables for production
9.1.5. Setup SSL/HTTPS certificate for backend API
9.1.6. Implement health check endpoints
9.1.7. Sprint review

9.2. Frontend Deployment
9.2.1. Setup static hosting (AWS S3, Vercel, Netlify)
9.2.2. Build and deploy optimized production frontend build
9.2.3. Configure custom domain and SSL for frontend
9.2.4. Setup redirect rules for client-side routing
9.2.5. Verify all API calls point to production backend
9.2.6. Sprint demo

9.3. CI/CD Pipeline
9.3.1. Configure automated build and test pipeline (GitHub Actions, GitLab CI)
9.3.2. Setup automated deployment to staging environment

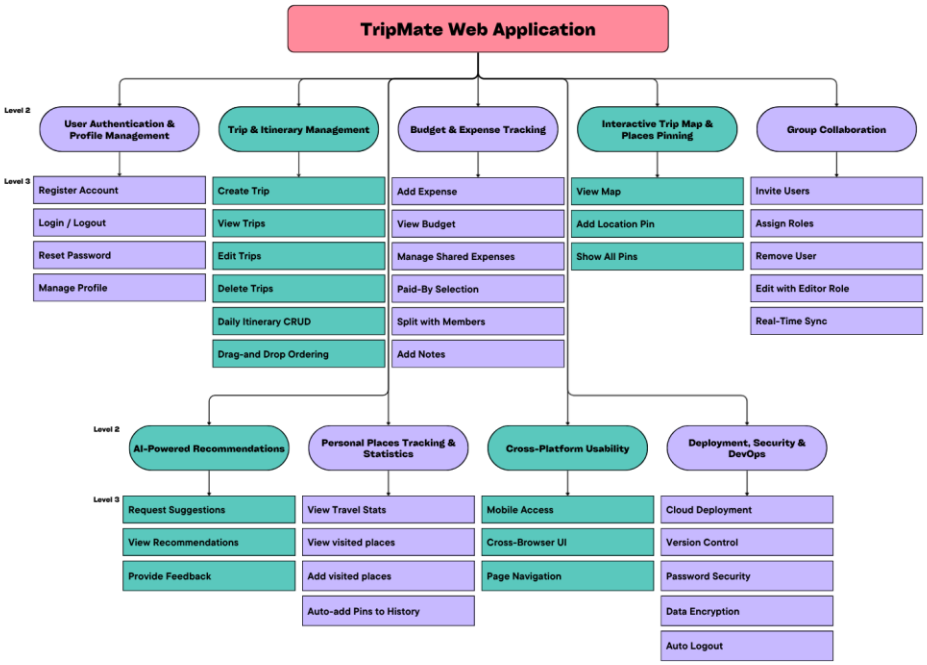
9.3.3. Implement manual approval gates for production deployment
9.3.4. Configure automated database migrations
9.3.5. Unit test deployment pipeline
9.3.6. Sprint review

9.4. Security & Monitoring
9.4.1. Implement input validation and sanitization across all APIs
9.4.2. Configure JWT token expiration and refresh logic
9.4.3. Setup API rate limiting and DDoS protection
9.4.4. Implement logging and error tracking (Winston, Sentry)
9.4.5. Configure automated database backups
9.4.6. Perform security audit and penetration testing
9.4.7. Sprint review & security sign-off

How This Works in Agile

- **Level 1 & Level 2** = Helps both the team and stakeholders understand the overall scope.
- **Level 3** = converts those features (Epics) into actionable **User Stories** that populate the product backlog.
- **Level 4** = breaks the user stories into **Sprint Tasks** during Sprint Planning.
- The **WBS is iterative**, meaning the team refines and expands it continuously as they learn more throughout development—not everything is defined upfront.

[WBS Diagram]



6.2 Milestones & Acceptance Criteria

Milestone	Time Frame	Purpose	Acceptance Criteria
M1: Project Kickoff & UX/UI Alignment	**Week 1-2**	Align the team on project goals and finalize UX/UI direction	- SRS is approved by all team members and faculty - UX/UI flow and wireframes for all major screens (Login, Trips, Itinerary, Budget, Map) are finalized - Design system defined (colors, typography, component style) - Technical architecture + technology stack agreed upon - Project backlog and sprint tasks created in GitHub milestone
M2: Sprint 2 Goal – Authentication & Profile Setup	**Week 3-4**	Deliver secure user account creation, access, and management	- Users can register and log in; system confirms login within 3 seconds - Passwords are stored using secure hashing - Account recovery via email is functional - Users can view and edit basic profile information
M3: Key Feature Delivery – Trips & Itinerary Management	**Week 5-6**	Build core trip creation and daily itinerary planning functionality.	- Users can create, view, edit, delete trips; system confirms creation/deletion within 2 seconds - Users can add/delete and sequence itinerary items and place pins - The itinerary supports up to 100 activities and maintains order on reload - Backend API successfully stores and retrieves all trip and itinerary data - Validation (no empty trip names, correct date ranges) implemented
M4: Incremental Release v1.0 – Budget & Map Integration	**Week 7-8**	Deliver first internal MVP (Trips + Itinerary + Budget + Map)	- Budget tracker supports adding/deleting expenses; total recalculates and updates automatically - Expense entries reflect on-screen within 2 seconds - Interactive map is integrated (Google maps) - Users can pin locations (up to 50 per trip) and pins are persistent on reload - Basic navigation between all core screens is functional
M5: Sprint 5 Goal – Group	**Week 9-10**	Transform the app into a collaborative	- Trip Manager can invite users via email and assign roles (Editor/Viewer); invitation is sent within 10 seconds

Collaboration & Real-Time Updates		tool with shared access and live sync	- Role-based permissions are enforced (e.g., Viewers cannot edit the itinerary/budget) - Edits to the itinerary or budget sync across all collaborators' screens within 1 second - Expense splitting calculates shares to a precision of $0.01
M6: Customer Feedback Loop – Usability Testing	**Week 11-12**	Validate the collaborative and AI features with real users and gather feedback	- At least 6-10 users complete a test of the core collaborative and AI workflow - The AI recommendation feature generates at least 3 relevant suggestions based on tags within 3 seconds - Usability issues are documented and prioritized in the backlog - Feedback forms are collected and synthesized - Revised wireframes or improvements planned for next sprint
M7: Feature Completion – AI, Personal Stats & Cross-Platform	**Week 13-14**	Deliver the feature-complete application with intelligence, personalization, and refined quality	- AI module is fully integrated; users can provide feedback (Thumbs Up/Down) on suggestions - Personal travel stats dashboard correctly displays counts of visited countries, cities, and places - Profile page shows a world map with visited places pinned - All major bugs from previous sprints are resolved; performance and security criteria from SRS are met
M8: Final Release & Project Closure	**Week 15-16**	Prepare the final, polished application for production and conclude the project	- The application is successfully deployed to a public cloud environment - Final project documentation, including user manuals, is submitted - A final demo is presented and signed off by faculty/stakeholders - A project retrospective is completed, documenting lessons learned

Section 7 – Product Backlog & Implementation Schedule

7.1. User Authentication & Profile Management

7.1.1. User Registration

As a new user, I want to register for an account with my email and a secure password so that I can access TripMate.

Description

Allow new users to create a secure account by providing required personal information and credentials.

Acceptance Criteria

- Registration form includes email, password, and basic profile fields
- Password must meet minimum security requirements (8+ characters)
- System validates email uniqueness before account creation
- Successful registration redirects to login page
- Clear error messages for duplicate emails, weak passwords, or invalid inputs

Testing Scenarios

1. User enters valid email and strong password → account created successfully → redirects to login
2. User enters existing email → shows "Email already registered" error
3. User enters password under 8 characters → shows "Password must be at least 8 characters"
4. User submits form with missing required fields → inline validation errors appear

7.1.2. User Login

As a registered user, I want to securely log into my account so that I can access my personal trips and travel data.

Description

Authenticate registered users using email and password to grant access to their travel planning workspace.

Acceptance Criteria

- Login form accepts email and password credentials
- Invalid credentials display generic error message for security
- Successful authentication redirects to user's home page/dashboard
- User session persists across browser refreshes until explicit logout
- System automatically logs out users after 30 minutes of inactivity

Testing Scenarios

1. User enters correct credentials → navigates to home page with full access
2. User enters incorrect password → shows "Invalid email or password" message
3. User remains inactive for 30+ minutes → automatically logged out and redirected to login
4. User refreshes browser → remains logged in with session intact

7.1.3. User Logout

As a logged-in user, I want to securely log out of my account so that my information remains protected on shared devices.

Description

Provide a secure way for users to end their session and clear authentication tokens.

Acceptance Criteria

- Logout option is clearly visible in navigation when user is authenticated
- Clicking logout immediately clears all session tokens and cookies
- User is redirected to login page with logout confirmation message
- All protected routes become inaccessible after logout
- Back-button navigation after logout does not restore access

Testing Scenarios

1. User clicks logout → session cleared → redirected to login page
2. User attempts back-button after logout → protected pages show authentication required
3. User logs out on shared device → next user cannot access previous account

7.1.4. Password Reset

As a user who forgot my password, I want to request a password reset via email so that I can regain access to my account.

Description

Enable users to securely recover account access through email verification and password reset flow.

Acceptance Criteria

- Password reset form accepts registered email address
- System sends secure reset link to email addresses
- Reset link expires after 24 hours for security
- User can set new password through secure reset flow
- Successful reset allows immediate login with new credentials

Testing Scenarios

1. User enters registered email → receives reset email within 60 seconds
2. User clicks expired reset link → shows "Link expired, please request new reset"
3. User successfully resets password → can immediately login with new password

7.1.5. View Profile Information

As a user, I want to view my profile information so that I can see my personal details and travel statistics.

Description

Users can see their profile details including personal information and travel statistics.

Acceptance Criteria

- Profile page displays user's name, email, and avatar
- Travel statistics show counts of visited countries, cities, and places
- Profile information loads within 2 seconds
- Statistics update automatically when new places are added

Testing Scenarios

1. User views profile → all personal information displayed correctly
2. User has visited places → statistics show accurate counts
3. User adds new place → statistics update without page refresh

7.1.6. Edit Profile Information

As a user, I want to edit my profile information so that I can keep my details up to date.

Description

Users can update their personal profile information.

Acceptance Criteria

- User can update name and email
- Changes save successfully
- Invalid inputs show appropriate error messages
- Profile picture can be uploaded

Testing Scenarios

1. User updates name → change persists after save
2. User cancels edit → original information preserved

7.2. Trip & Itinerary Management

7.2.1. Create New Trip

As a user, I want to create a new trip by specifying destination and travel dates so that I can begin organizing my travel plans.

Description

Enable users to initialize new trips with basic information including destinations and dates.

Acceptance Criteria

- Trip creation form includes destination, start date, and end date
- Date validation ensures end date is not before start date
- Required fields must be completed before submission
- New trip saves successfully and appears in user's trip list

Testing Scenarios

1. User enters valid destination and date range → trip created successfully → appears in home page
2. User leaves destination empty → shows "Destination required" validation error
3. User selects end date before start date → shows "End date must be after start date" error
4. User refreshes page after creation → trip remains in list

7.2.2. View Trip

As a user, I want to see all my trips organized in a home page so that I can quickly access and manage my travel plans.

Description

Provide users with a centralized view of all their trips with quick access options.

Acceptance Criteria

- Home page displays list of user's trips formatted as cards
- Each trip card shows destination, dates, participants, and trip type (solo or group)
- Clicking a trip card navigates to the detailed trip view

Testing Scenarios

1. User with multiple trips → all display with correct information and sorting
2. User clicks on trip card → navigates to corresponding trip details page
3. User with no trips → sees default example trip card

7.2.3. Edit Trip Details

As a trip owner or editor, I want to modify trip information so that I can adjust plans as circumstances change.

Description

Allow authorized users to update trip details including destination, dates, and descriptions.

Acceptance Criteria

- Edit form pre-populates with current trip information
- Users can modify destination, dates, itinerary, budget and group members
- All validation rules from trip creation apply to edits
- Real-time updates propagate to all trip collaborators

Testing Scenarios

1. User opens trip for editing → sees current trip details pre-filled
2. User updates destination → change saves successfully and reflects in trip view
3. User attempts invalid date change → validation prevents save with clear error
4. Collaborator views trip after edit → sees updated information immediately

7.2.4. Add Itinerary Activity

As a trip planner, I want to add activities to my daily itinerary so that I can build a comprehensive travel schedule.

Description

Enable users to create new itinerary items

Acceptance Criteria

- Activity creation includes title
- Activities save to correct itinerary date
- New activities appear immediately in the itinerary view

Testing Scenarios

1. User adds activity → appears under correct day
2. User leaves required title empty → validation prevents save
3. User adds multiple activities → all display in chronological order
4. Activity appears in itinerary immediately after creation

7.2.5. Delete Itinerary Activity

As a trip planner, I want to remove activities from my itinerary so that I can eliminate unwanted or cancelled plans.

Description

Enable safe removal of itinerary items with appropriate cleanup.

Acceptance Criteria

- Activity removal happens immediately upon initiation
- Deletion propagates to all collaborators in real-time

Testing Scenarios

1. User deletes activity → item removed from itinerary immediately
2. Collaborator views itinerary after deletion → activity no longer appears
3. Multiple activities deleted → itinerary maintains proper sequence

7.2.6. Reorder Itinerary Items

As a trip planner, I want to drag and drop activities to reorder them so that I can optimize my daily schedule flow.

Description

Provide intuitive drag-and-drop interface for sequencing itinerary activities logically.

Acceptance Criteria

- Drag handles or entire items are draggable
- Visual feedback shows drop targets and positioning
- New order saves automatically without manual save action
- Order persists across page refreshes and collaborator views

Testing Scenarios

1. User drags activity to new position → order updates immediately
2. User refreshes page → maintained order persists
3. Multiple items reordered → all maintain correct sequence
4. Collaborator views reordered itinerary → sees updated sequence

7.2.7. Delete Entire Trip

As a trip owner, I want to permanently delete a trip so that I can remove plans I no longer need.

Description

Provide secure trip deletion with appropriate confirmation and data cleanup.

Acceptance Criteria

- Delete option available only to trip owners
- Deletion requires explicit confirmation due to destructive nature
- Trip and all associated data removed from database
- Trip disappears from all collaborators' views immediately

Testing Scenarios

1. Trip owner deletes trip with confirmation → trip removed from all views
2. Non-owner attempts deletion → option unavailable or permission denied
3. User cancels deletion confirmation → trip remains intact

7.3. Budget & Expense Tracking

7.3.1. Add Expense to a Trip

As a user, I want to add an expense (name, amount) to my trip so that I can track my spending.

Description

Users need the ability to input expenses during trip planning or while travelling. Each expense requires a name, and amount. This helps users maintain accurate budgeting and spending oversight.

Acceptance Criteria

- User can enter the expense name.
- User can enter the amount (must validate numeric input).
- Expense is saved to the correct trip.
- Expense appears immediately in the expense list.

Testing Scenarios

1. User enters valid expense values → system saves and displays in list
2. User enters invalid amount → shows "Invalid amount" error
3. Empty name field → error message appears

7.3.2. View Total Trip Expenses

As a user, I want to view the total trip expense so that I can monitor my overall spending.

Description

Users require a dynamic budget display that sums all expenses automatically. This helps them understand spending totals immediately.

Acceptance Criteria

- System calculates total expenses automatically.
- Total updates when expenses are added/deleted.
- Total is formatted properly (e.g., $123.45).
- Total is visible on left part of the budget page.

Testing Scenarios

1. Add several expenses → total should update
2. Delete an expense → total decreases
3. Edit an amount → total recalculates instantly
4. Page reload → total remains accurate

7.3.3. Mark an Expense as Shared

As a group traveller, I want to make an expense as “shared” so that the system can split it among participants.

Description
Expenses may be shared among members in a group trip. The user should be able to mark an item as shared.
Acceptance Criteria

- Expense has a checkbox for “Shared”.
- Checking the box marks expense as shared.
- State persists after page reload.
- Checkbox disabled for users without edit permissions.
- Checkbox fades if trip has no group.

Testing Scenarios

1. Check the Shared checkbox → shared state appears
2. Uncheck the Shared checkbox → shared state removed
3. Refresh page → status preserved
4. User without edit rights cannot modify shared status
5. No group → Shared checkbox fade

7.3.4 Select Who Paid an Expense
As a group traveler, I want to select who paid an expense so that the payer is tracked correctly for splitting.
Description
Users can select a payer from the group for proper reimbursement calculations. Only one payer can be chosen per expense.
Acceptance Criteria

- Dropdown shows all group members.
- One member can be selected as payer.
- Payer information is stored in the database.
- Payer name is shown in the Expense Details view.
- When the payer changes, per-person calculations update instantly.

Testing Scenarios

1. Select payer from dropdown → correct name listed
2. Switch payer → update reflected immediately
3. Refresh page → payer remains saved
4. Remove member from group → they disappear from payer dropdown
5. User without edit rights → dropdown disabled

7.3.5. Select Group Members to Split Cost
As a group traveler, I want to select which group members will split a shared expense so that the cost is divided fairly.
Description
Users choose participants for splitting. The system divides the total equally.
Acceptance Criteria

- Multi-select list of group members displayed.
- System automatically calculates per-person split.
- Selected members stored with the expense.
- UI displays individual share amounts.

Testing Scenarios

1. Select *n* members → amount splits equally

2. Select or deselect a member → recalculation happens
3. Member removed from trip → removed from split list
4. Page reload → saved selections preserved

7.3.6. Add Notes to Shared Expenses

As a group traveler, I want to add notes to shared expenses so that I can record extra details.

Description

Notes help clarify context (e.g., "Paid cash", "Birthday dinner").

Acceptance Criteria

- Text area available for optional notes.
- Notes saved with expense.
- Notes visible in expanded expense view.

Testing Scenarios

1. Enter note → save → re-open → note is present
2. Edit note → new version overwrites old
3. Leave note blank → expense still saves

7.4. Interactive Trip Map & Places Pinning

7.4.1. View Interactive Map

As a user, I want to view an interactive map centered on my trip destination so that I can explore the area visually.

Description

A dynamic map must load and automatically center on the trip's destination coordinates.

Acceptance Criteria

- Map loads using chosen provider (Google Maps/Leaflet/Mapbox).
- Map centers on destination.
- Zoom controls available.
- Map loads within 3 seconds.

Testing Scenarios

1. Destination = "Paris" → map centers on Paris
2. Slow network → map still loads with fallback message
3. Zoom in/out works

7.4.2. Pin Locations to Trip Map

As a user, I want to pin locations (restaurants, attractions) so that I can visualize them on my trip map.

Description

Users can attach pins on the map to visualize their itinerary.

Acceptance Criteria

- User can click on Add New Pin button to add a pin
- Pin saved with coordinates
- Pin displayed immediately

Testing Scenarios

1. Save pin → displayed instantly
2. Delete pin → updates shown
3. Reload → pins remain persisted

4. More than 50 pins -> system prevents adding new

7.4.3. Display All Pinned Locations

As a user, I want all my pinned locations displayed at once so that I can get a geographic overview of my trip.

Description

Map should show all pins simultaneously.

Acceptance Criteria

- All saved pins auto-load on page open
- Pins have consistent styling
- Map automatically adjusts zoom to fit all pins

Testing Scenarios

1. 0 pins → map shows default view
2. Many pins → map clusters or zoom adjusts
3. Reload → all pins remain

7.5. Group Collaboration

7.5.1. Invite Users to Trip

As a Trip Manager, I want to invite users to a trip via their email address so that they can collaborate with me.

Description

Trip managers need to invite other travelers to join trip planning. This enables shared itinerary development and group coordination.

Acceptance Criteria

- Email invitation system functional
- Invitation link/email sent successfully
- Invited users can join trip
- Invitation status tracked (pending/accepted/declined)

Testing Scenarios

1. Send invitation → email received by recipient
2. Accept invitation → user added to trip

7.5.2. Assign User Roles

As a Trip Manager, I want to assign a role (Editor or Viewer) to a user when inviting them so that I can control their permissions.

Description

Different permission levels ensure appropriate access control. Editors can modify trip details while Viewers can only see information.

Acceptance Criteria

- Role selection during invitation (Editor/Viewer)
- Role-based permissions enforced throughout application
- Different access levels for Editors vs Viewers
- Role changes logged for audit purposes

Testing Scenarios

1. Assign Editor role → user can edit trip details
2. Assign Viewer role → user can only view trip data

3. Change role → permissions update immediately

7.5.3. Remove User from Trip

As a Trip Manager, I want to remove a user from a trip so that I can manage the group.

Description

Trip managers need control over group membership to maintain appropriate collaboration and remove inactive or unwanted participants.

Acceptance Criteria

- Remove user functionality available to trip managers
- User access revoked immediately upon removal
- Data permissions updated across all trip components

Testing Scenarios

1. Remove user → access immediately revoked
2. Removed user tries to access trip → permission denied
3. Multiple removals → system handles correctly

7.5.4. Edit Shared Trip Details

As a collaborator with Editor access, I want to edit the details of an existing trip so that I can adjust shared plans.

Description

Editors should be able to modify trip information to contribute to collaborative planning while maintaining data integrity.

Acceptance Criteria

- Editors can modify trip destination, dates, itinerary items and trip expenses
- Same item cannot be edited by multiple users simultaneously
- Locking mechanism prevents concurrent edits on the same element
- Visual indicators show when other users are editing specific items
- Changes visible to all collaborators in real-time
- Permission checks prevent unauthorized edits
- Edit history tracked for accountability

Testing Scenarios

1. Editor edits trip destination → changes saved successfully
2. User A editing expense → User B sees "User A is editing" indicator
3. User B tries to edit same expense → receives "item locked" message
4. User A finishes editing → lock released and item available for others
5. Viewer tries to edit → permission denied with error message

7.5.5. Real-time Trip Updates

As any collaborator, I want to see real-time updates made by others on the trip board so that we are always in sync.

Description

Real-time synchronization ensures all group members see the same information simultaneously, preventing version conflicts and miscommunication.

Acceptance Criteria

- Real-time synchronization implemented across all trip data
- Changes appear within 1 second to all collaborators
- No page refresh required to see updates
- System handles network interruptions gracefully

Testing Scenarios

1. User A edits trip details → User B sees change immediately
2. Multiple users viewing same trip → all see consistent, up-to-date information
3. Network interruption → sync resumes when connection restored

7.6. AI-Powered Recommendations

7.6.1. Request AI Recommendations by Tag

As a user, I want to request recommendations for places/activities based on tags (e.g., #food, #sightseeing) so that I can discover new ideas.

Description

Users should be able to choose one or more tags and receive personalized suggestions generated by the AI engine. This helps users quickly find relevant travel ideas.

Acceptance Criteria

- User can select one or more tags from a predefined list.
- System generates at least 5 relevant recommendations per tag.
- Recommendations include name, location, short description, and category tag.
- Error message appears if no recommendations are available.

Testing Scenarios

1. User selects “#food” → system returns food-related places.
2. User selects multiple tags (#food + #nature) → combined suggestions returned.
3. User selects a tag with no data → system shows “No recommendations available.”
4. User requests recommendations with poor connection → loading indicator appears.

7.6.2. View AI Recommendations

As a user, I want to view AI-generated recommendations in the chat response so that I can get suggestions for my trip planning.

Description

AI assistant provides travel suggestions as formatted response messages within the chat interface.

Acceptance Criteria

- Recommendations display as part of the AI chat response message
- Response includes place names, descriptions, and reasoning in readable format
- Message formatting maintains clear structure and readability
- System handles cases where no recommendations are available
- Response appears within 3 seconds of request

Testing Scenarios

1. User requests recommendations → sees formatted suggestions in AI response message
2. AI finds no matching results → response message says "No recommendations found"
3. User receives multiple suggestions → all display in well-formatted chat response
4. Request times out → appropriate error message appears in chat

7.6.3. Provide Feedback on Recommendation

As a user, I want to provide positive or negative feedback on a recommendation so that the AI can learn and improve.

Description

User should be able to rate generated recommendations. Data will improve next iteration of recommendation quality.

Acceptance Criteria

- Each recommendation has a 👍 or 👎 option.
- Feedback sends asynchronously without reloading the page.
- User can only vote once per recommendation.
- System stores feedback for future analysis.

Testing Scenarios

1. User clicks 👍 → feedback recorded, system notes preference
2. User clicks 👎 → same response regenerates.
3. Feedback is still saved after the user refreshes the page.

7.7. Personal Places Tracking & Statistics

7.7.1. View Personal Travel Statistics

As a user, I want to see a Personal Traveling Statistics section on my profile showing my total number of visited countries, cities, and places so that I can track my travel achievements.

Description

The profile page should display simple quantitative stats about the user's travel history.

Acceptance Criteria

- Stats section displays: total countries, total cities, total places.
- Numbers increase automatically as new places are added.
- Data updates instantly without page reload.
- If no places exist, system shows 0s

Testing Scenarios

1. User with 0 places → stats show all zeros.
2. User adds a new place → stats increase accordingly.
3. User deletes a place → stats decrease.
4. Stats load correctly even after refreshing.

7.7.2. View Personal Travel Map

As a user, I want to view a world map on my profile page where all my visited places are pinned so that I can visually relive my travels.

Description

A world map visualization with interactive pins helps users see their travel history.

Acceptance Criteria

- Map loads on profile page.
- All visited places appear as pins with tooltip (name + location).
- Clicking a pin show expanded info.
- Zoom and pan supported on desktop and mobile.

Testing Scenarios

1. User has *n* visited places → all show as pins.
2. User zooms and pans → map behaves smoothly.
3. No visited places → map shows default view

7.7.3. Manually Add a Visited Place

As a user, I want to manually add a visited place to my profile map to include travels not planned in TripMate.

Description

Users may have trips outside the app. They should be able to manually add specific visited locations.

Acceptance Criteria

- "Add Pin" button appears on the map.
- New place appears immediately after saving.
- Stats update automatically.

Testing Scenarios

1. Added place pin shows on map + updates stats.
2. User adds multiple places → map update correctly.

7.7.4. Auto-Add Itinerary Places to Personal History

As a user, I want all trip's pinned itinerary locations to be automatically added to my personal history map and stats when I create a new trip.

Description

Users don't need to manually add places that already exist in a trip. The places from itinerary should sync automatically.

Acceptance Criteria

- When user adds a location to a trip itinerary, it is also stored in the visited places database.
- Map updates without manual action.
- Duplicate places are not added twice.
- Stats increase accordingly.

Testing Scenarios

1. User adds *n* places in itinerary → all appear on profile map.
2. User removes a place from itinerary → place is removed from visited history.
3. System prevents adding the same coordinates twice.
4. Stats match the number of unique places.

7.8. Cross-Platform Usability

7.8.1. Mobile Browser Support

As a mobile user, I want to use all core TripMate features on my phone's browser so that I can plan on the go.

Description

The application must remain responsive and usable on mobile devices.

Acceptance Criteria

- All main pages are responsive.
- Navigation menu adapts to mobile layout.
- Buttons and interactive elements meet touch-target standards.
- Map and itinerary features remain usable on mobile.

Testing Scenarios

1. User opens app on iPhone → all pages scale correctly.
2. Mobile menu opens and closes properly.
3. Adding pins on map works on mobile.
4. Budget page scrolls correctly on small screens.

7.8.2. Cross-Browser Compatibility

As a user, I want to experience a consistent and intuitive interface across Chrome, Firefox, and Safari.

Description

UI and core features should function the same across major browsers.

Acceptance Criteria

- No layout failures on Chrome, Firefox, Safari.
- All interactive elements behave consistently.
- Map rendering works across all browsers.
- Fonts, spacing, and components maintain consistency.

Testing Scenarios

1. User loads TripMate on Chrome → no visual issues.
2. User loads TripMate on Safari (iOS) → layout identical.
3. Firefox user clicks buttons, opens modals → all functional.
4. Browser zoom (90–125%) does not break layout.

7.8.3. Navigation Across Designed Pages

As a user, I want to navigate through the web application using the designed pages (Landing, Home, Profile, Destination, Itinerary, Budget, Group).

Description

Navigation should be simple, predictable, and consistent across all devices.

Acceptance Criteria

- Navigation bar includes links to all pages.
- Clicking a link loads the correct page without delay.
- Breadcrumbs or back buttons provided where needed.
- Navigation elements remain visible and responsive.

Testing Scenarios

1. User clicks “Profile” → profile page loads.
2. User moves from Destination → Itinerary → Budget → Group page smoothly.
3. Navigation persists across mobile and desktop layouts.
4. Invalid page routes redirect to a 404-friendly screen.

7.9. Deployment, Security & DevOps

7.9.1. Deploy to Cloud Platform

As a developer, I want to deploy the web application to a cloud platform (e.g., AWS) so that it is publicly accessible.

Description

The application needs reliable cloud hosting with proper infrastructure for public access and scalability.

Acceptance Criteria

- Application deployed to cloud provider (AWS/Azure/Google Cloud)
- Public URL accessible worldwide
- SSL certificate configured
- Environment variables properly set

Testing Scenarios

1. Access public URL → application loads correctly
2. HTTPS enforced → secure connection established
3. Different regions → application accessible globally

7.9.2. Version-Controlled Codebase

As a developer, I want to have a version-controlled codebase (GitHub) and a defined process for integrating code.

Description

Proper version control and CI/CD processes ensure code quality and team collaboration efficiency.

Acceptance Criteria

- GitHub repository established with proper branching strategy
- Pull request process defined and enforced
- Continuous integration pipeline configured
- Code review requirements established

Testing Scenarios

1. Create pull request → automated tests run
2. Merge to main → automatic deployment triggered
3. Code conflicts → proper resolution process followed

7.9.3. Password Hashing & Salting

As a system, I want to (must) hash and salt user passwords before storing them in the database so that data is secure.

Description

User credentials must be protected using industry-standard security practices to prevent data breaches.

Acceptance Criteria

- Passwords hashed using bcrypt or similar algorithm
- Unique salt generated for each password
- Plain text passwords never stored or logged
- Hash verification works for login authentication

Testing Scenarios

1. User registers → password stored as hash only
2. User logs in → hash verification succeeds with correct password
3. Database compromised → passwords cannot be reversed

7.9.4. Data Encryption

As a system, I want to (must) encrypt sensitive data in transit and at rest to protect user privacy.

Description

All sensitive user data must be encrypted to meet privacy standards and protect against unauthorized access.

Acceptance Criteria

- HTTPS/TLS encryption for all data in transit
- Database encryption for sensitive data at rest
- API responses encrypted
- Secure key management implemented

Testing Scenarios

1. Network traffic analysis → data encrypted
2. Database file access → sensitive fields encrypted
3. API calls → encrypted payloads

7.9.5. Session Timeout

As a system, I want to (must) log users out after 30 minutes of inactivity to prevent unauthorized access.

Description

Automatic session termination protects users who forget to log out, especially on shared devices.

Acceptance Criteria

- 30-minute inactivity timer implemented
- User notified before automatic logout
- Session properly terminated and tokens invalidated
- Graceful redirect to landing page

Testing Scenarios

1. 30 minutes inactive → automatic logout
2. User activity resets timer → session continues
3. Logged out user tries to access protected route → redirected to login

7.10. Additional Technical & Supporting Stories

7.10.1. Database Schema Migration Setup

As a developer, I want to (need to) have database migration scripts so that schema changes can be version-controlled and deployed safely.

Description

Implement database migration system for managing schema evolution.

Acceptance Criteria

- Migration scripts for all initial tables created
- Rollback scripts available for each migration
- Migration system integrated with deployment pipeline
- Success/failure logging implemented

Testing Scenarios

1. Run initial migration → all tables created successfully
2. Run rollback → schema reverts correctly
3. Deploy to new environment → migrations run automatically

7.10.2. API Documentation Generation

As a developer, I want to (need to) have automated API documentation so that frontend and backend teams can understand available endpoints.

Description

Generate and maintain API documentation automatically from code.

Acceptance Criteria

- OpenAPI documentation generated
- All endpoints documented with parameters and responses
- Documentation accessible via web interface
- Updates to API auto-update documentation

Testing Scenarios

1. Add new endpoint → appears in documentation
2. Modify existing endpoint → documentation updates
3. Access docs URL → complete API reference loads

7.10.3. Performance Testing Setup

As a quality assurance engineer, I want to (need to) have performance tests so that I can verify system meets performance requirements.

Description

Implement performance testing for critical user flows.

Acceptance Criteria

- Response time measurements for key actions
- Performance baseline established
- Automated performance regression detection

Testing Scenarios

1. Run load test → verify <2s response times under load
2. Performance regression → automated alert triggered

7.10.4. Security Vulnerability Scanning

As a security engineer, I want to (need to) have automated security scans so that vulnerabilities can be detected early.

Description: Implement security scanning in CI/CD pipeline.

Acceptance Criteria

- Dependency vulnerability scanning
- Code security analysis
- Automated security testing in pipeline
- Security reports generated for each build

Testing Scenarios

1. Push code with vulnerable dependency → build fails with security warning
2. Fix vulnerability → build passes security scan
3. Monthly security audit → comprehensive report generated

7.10.5. Application Monitoring & Alerting

As an operations engineer, I want to (need to) have application monitoring so that I can detect and respond to issues proactively.

Description

Set up monitoring, logging, and alerting for production system.

Acceptance Criteria

- Application performance monitoring configured
- Error tracking and alerting implemented
- Database performance metrics collected
- Uptime monitoring with alerts

Testing Scenarios

1. Application error occurs → alert sent to team
2. Performance degrades → monitoring detects and reports
3. Database slow query → logged and alerted

Section 8 – Client/Faculty Sign-off

This section confirms that the TripMate Software Requirements Specification (SRS) has been reviewed and formally approved by all stakeholders, including the project team and the course instructor. By signing below, each party acknowledges that the document accurately reflects the agreed-upon requirements and will serve as the official baseline for all subsequent design, development and testing activities.

Approval Signature

Name	Role	Date	Signature
Yasser Elmankabady (PRJ566)	Course Instructor	2025-12-14	_____
Janice Kang	Backend Developer	2025-12-14	_____
Uny Li	Cloud & Security Engineer	2025-12-14	_____
Seliya Marahatta	AI & Data Analytics Engineer	2025-12-14	_____
Diliara Khairullina	Project Manager & Frontend Developer	2025-12-14	

Acknowledgment

By signing this document, all parties agree that:

1. The requirements outlined in this SRS are complete, accurate and reflect the current understanding of project goals.
2. Any future modifications will follow the documented **Change Control Process**.
3. This SRS serves as authoritative foundation for all upcoming phases including system architecture, implementation and testing.